

# User Manual

for S32 UART Driver

Document Number: UM11UARTASR4.4 Rev0000R4.0.2 Rev. 1.0

|                                                                                        |           |
|----------------------------------------------------------------------------------------|-----------|
| <b>1 Revision History</b>                                                              | <b>2</b>  |
| <b>2 Introduction</b>                                                                  | <b>3</b>  |
| 2.1 Supported Derivatives                                                              | 3         |
| 2.2 Overview                                                                           | 4         |
| 2.3 About This Manual                                                                  | 4         |
| 2.4 Acronyms and Definitions                                                           | 5         |
| 2.5 Reference List                                                                     | 5         |
| <b>3 Driver</b>                                                                        | <b>7</b>  |
| 3.1 Requirements                                                                       | 7         |
| 3.2 Driver Design Summary                                                              | 7         |
| 3.3 Hardware Resources                                                                 | 8         |
| 3.3.1 Physical Uart Channels:                                                          | 8         |
| 3.3.2 Uart pins:                                                                       | 8         |
| 3.4 Deviations from Requirements                                                       | 8         |
| 3.5 Driver Limitations                                                                 | 9         |
| 3.5.1 Stop bits limitation                                                             | 9         |
| 3.5.2 Linflexd_0 limitation                                                            | 9         |
| 3.6 Driver usage and configuration tips                                                | 9         |
| 3.6.1 How to use User notifications                                                    | 9         |
| 3.6.2 How to configure multicore                                                       | 12        |
| 3.6.3 How to configure DMA                                                             | 14        |
| 3.6.4 How to choose the appropriate timeout in general                                 | 19        |
| 3.6.5 How to enable Timeout Interrupt to monitor the idle state of the reception line. | 19        |
| 3.7 Runtime errors                                                                     | 20        |
| 3.8 Symbolic Names Disclaimer                                                          | 21        |
| <b>4 Tresos Configuration Plug-in</b>                                                  | <b>22</b> |
| 4.1 Module Uart                                                                        | 23        |
| 4.2 Container GeneralConfiguration                                                     | 23        |
| 4.3 Parameter UartDevErrorDetect                                                       | 24        |
| 4.4 Parameter DisableUartRuntimeErrorDetect                                            | 24        |
| 4.5 Parameter UartMulticoreSupport                                                     | 25        |
| 4.6 Parameter UartEnableUserModeSupport                                                | 25        |
| 4.7 Parameter UartTimeoutMethod                                                        | 26        |
| 4.8 Parameter UartTimeoutDuration                                                      | 26        |
| 4.9 Parameter UartDmaEnable                                                            | 27        |
| 4.10 Parameter UartVersionInfoApi                                                      | 27        |
| 4.11 Parameter UartCallbackCapability                                                  | 28        |
| 4.12 Parameter UartCallback                                                            | 28        |

|                                                      |           |
|------------------------------------------------------|-----------|
| 4.13 Reference UartEcucPartitionRef . . . . .        | 30        |
| 4.14 Container UartGlobalConfig . . . . .            | 30        |
| 4.15 Container UartChannel . . . . .                 | 31        |
| 4.16 Parameter UartChannelId . . . . .               | 31        |
| 4.17 Parameter UartHwChannel . . . . .               | 32        |
| 4.18 Reference UartClockRef . . . . .                | 32        |
| 4.19 Reference UartChannelEcucPartitionRef . . . . . | 33        |
| 4.20 Container DetailModuleConfiguration . . . . .   | 33        |
| 4.21 Parameter DesireBaudrate . . . . .              | 34        |
| 4.22 Parameter UartInteruptDmaMethod . . . . .       | 34        |
| 4.23 Parameter UartParityEnable . . . . .            | 35        |
| 4.24 Parameter UartParityType . . . . .              | 35        |
| 4.25 Parameter UartStopBitNumber . . . . .           | 36        |
| 4.26 Parameter UartWordLength . . . . .              | 36        |
| 4.27 Parameter UartTimeoutEnable . . . . .           | 37        |
| 4.28 Reference UartDmaTxChannelRef . . . . .         | 37        |
| 4.29 Reference UartDmaRxChannelRef . . . . .         | 38        |
| 4.30 Container CommonPublishedInformation . . . . .  | 38        |
| 4.31 Parameter ArReleaseMajorVersion . . . . .       | 39        |
| 4.32 Parameter ArReleaseMinorVersion . . . . .       | 39        |
| 4.33 Parameter ArReleaseRevisionVersion . . . . .    | 40        |
| 4.34 Parameter ModuleId . . . . .                    | 40        |
| 4.35 Parameter SwMajorVersion . . . . .              | 41        |
| 4.36 Parameter SwMinorVersion . . . . .              | 41        |
| 4.37 Parameter SwPatchVersion . . . . .              | 42        |
| 4.38 Parameter VendorApiInfix . . . . .              | 42        |
| 4.39 Parameter VendorId . . . . .                    | 44        |
| <b>5 Module Index . . . . .</b>                      | <b>45</b> |
| 5.1 Software Specification . . . . .                 | 45        |
| <b>6 Module Documentation . . . . .</b>              | <b>46</b> |
| 6.1 Linflexd UART IPL . . . . .                      | 46        |
| 6.1.1 Detailed Description . . . . .                 | 46        |
| 6.1.2 Data Structure Documentation . . . . .         | 47        |
| 6.1.3 Types Reference . . . . .                      | 49        |
| 6.1.4 Enum Reference . . . . .                       | 49        |
| 6.1.5 Function Reference . . . . .                   | 51        |
| 6.2 UART Driver . . . . .                            | 60        |
| 6.2.1 Detailed Description . . . . .                 | 60        |
| 6.2.2 Data Structure Documentation . . . . .         | 61        |



|                                    |    |
|------------------------------------|----|
| 6.2.3 Enum Reference . . . . .     | 61 |
| 6.2.4 Function Reference . . . . . | 63 |



## Chapter 1

### Revision History

| Revision | Date       | Author       | Description                                             |
|----------|------------|--------------|---------------------------------------------------------|
| 1.0      | 30.06.2023 | NXP RTD Team | S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release |

## Chapter 2

### Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor CDD UART for S32 microcontrollers. CDD UART driver configuration parameters and deviations from the specification are described in Driver chapter of this document.

### 2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a\_bga525
- s32g254a\_bga525
- s32g233a\_bga525
- s32g234m\_bga525
- s32g378a\_bga525
- s32g379a\_bga525
- s32g398a\_bga525
- s32g399a\_bga525
- s32g338m\_bga525
- s32g339m\_bga525
- s32g358a\_bga525
- s32g359a\_bga525
- s32r45\_bga780

All of the above microcontroller devices are collectively named as S32.

## 2.2 Overview

**AUTOSAR (AUTomotive Open System ARchitecture)** is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

## 2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

## 2.4 Acronyms and Definitions

| Term       | Definition                                  |
|------------|---------------------------------------------|
| API        | Application Programming Interface           |
| AUTOSAR    | Automotive Open System Architecture         |
| BSMI       | Basic Software Make file Interface          |
| BSW        | Basic Software                              |
| CS         | Chip Select                                 |
| DEM        | Diagnostic Event Manager                    |
| DET        | Development Error Tracer                    |
| DMA        | Direct Memory Access                        |
| ECU        | Electronic Control Unit                     |
| FIFO       | First In First Out                          |
| GUI        | Graphical User Interface                    |
| ISR        | Interrupt Service Routine                   |
| LSB        | Least Significant Bit                       |
| LT Variant | Link Time Variant                           |
| MCU        | Micro Controller Unit                       |
| MIDE       | Multi Integrated Development Environment    |
| MSB        | Most Significant Bit                        |
| N/A        | Not Applicable                              |
| OS         | Operating System                            |
| UART       | Universal asynchronous receiver-transmitter |
| PB Variant | Post Build Variant                          |
| PC Variant | Pre Compile Variant                         |
| RAM        | Random Access Memory                        |
| RTD        | Real Time Drivers                           |
| SIU        | Systems Integration Unit                    |
| SWS        | Software Specification                      |
| XML        | Extensible Markup Language                  |

## 2.5 Reference List

| #  | Title                                 | Version               |
|----|---------------------------------------|-----------------------|
| 1  | S32G2 Reference Manual                | Rev. 7, February 2023 |
| 2  | S32G3 Reference Manual                | Rev.3, Feb 2023       |
| 3  | S32R45 Reference Manual               | Rev. 3, 12/2021       |
| 4  | S32G2: Mask Set Errata for Mask 0P77B | Rev. 3.0              |
| 5  | S32G2: Mask Set Errata for Mask 1P77B | Rev. 1.0              |
| 6  | S32G3: Mask Set Errata for Mask 1P72B | Rev. 2.1              |
| 7  | S32R45: Mask Set Errata for Mask P57D | Rev. 2.0              |
| 8  | S32G2 Data Sheet                      | Rev 6, Dec 2022       |
| 9  | S32G3 Data Sheet                      | Rev 2, RC, Feb 2023   |
| 10 | VR5510 Data Sheet                     | Rev 5, April 2022     |



## Introduction

| #  | Title             | Version              |
|----|-------------------|----------------------|
| 11 | S32R45 Data Sheet | Rev. 3, RC — 11/2022 |

## Chapter 3

### Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

### 3.1 Requirements

UART is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

### 3.2 Driver Design Summary

The Uart driver is part of the Real Time Drivers, performs the hardware access and offers a hardware independent API to the application. The Uart driver is implemented as an Autosar complex device driver. It uses the Uart hardware peripheral which provides support for implementing the Uart protocol. Hardware and software settings can be configured using an Autosar standard configuration tool. The driver reports errors to the error manager as defined in AUTOSAR. The following features are implemented in the Uart Module for S32G and S32R devices:

- Full-duplex communication
- Configurable baud-rate. It supports the standard UART rates. The values can be checked in the Uart\_↔BaudrateType enum.
- Configurable parity, stop bits
- Usage in a multicore enviroment
- DMA transfers
- Callback notifications
- Continuous transfers in an asynchronous manner

### 3.3 Hardware Resources

**3.3.1 Physical Uart Channels:** The hardware instances configured by the Uart driver:

- For all devices of the S32G2/S32G3 family have 3 channels: LINFLEXD\_0, LINFLEXD\_1, LINFLEXD\_2.
- For all devices of the S32R45 family have 2 channels: LINFLEXD\_0, LINFLEXD\_1. **Note:** In EB tresos, Uart Physical Unit has selected by UartHwChannel ("General" tab).

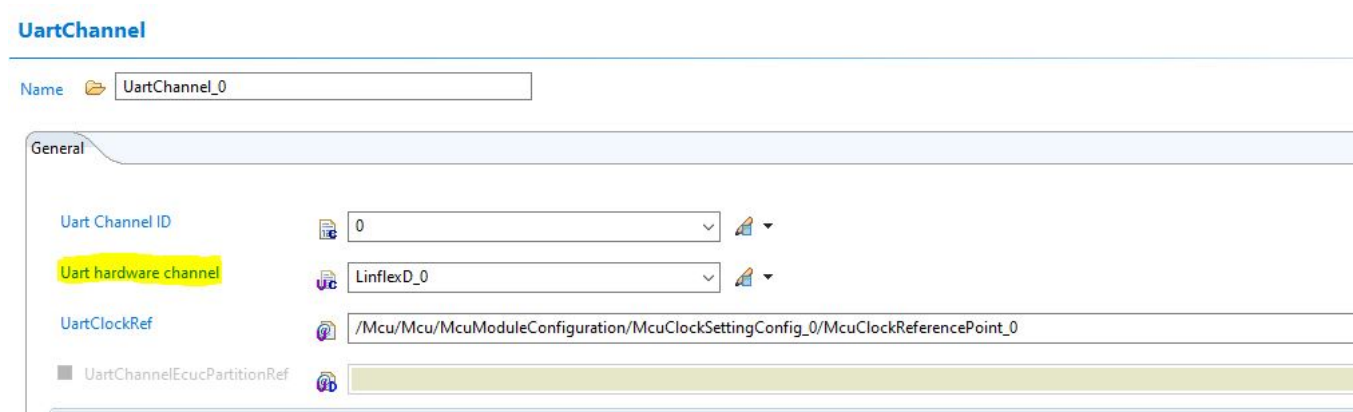


Figure 3.1 Uart Physical Unit has selected by UartPhyUnitMapping in EB Tresos

**3.3.2 Uart pins:** The Uart\_0, Uart\_1, Uart\_2 use the pins Lin\_0, Lin\_1, Lin\_2 correspondingly. The pins can be found in the file "S32G274A\_IOMUX\_V02.xlsx", "S32G3\_IOMUX.xlsx" and "S32R45\_IOMUX.xlsx" from attached file of S32G274 Reference Manual, S32G3 Reference Manual and S32R45 Reference Manual respectively.

| Port  | Pair Port | CR <sup>1</sup> | CR Instance Name | SSS       | Addr       | Function <sup>2</sup> | Module     | Description       | Di |
|-------|-----------|-----------------|------------------|-----------|------------|-----------------------|------------|-------------------|----|
| PA_13 |           |                 |                  | 0000_0010 |            | LIN1_TX               | LINFLEXD_1 | LIN Transmit Data | 0  |
| PA_14 |           |                 | 512 SIUL2_0      | 0000_0011 | 0x4009CA40 | LIN0_RX               | LINFLEXD_0 | LIN Receive Data  | 1  |
| PA_15 |           |                 |                  | 0000_0010 |            | LIN0_TX               | LINFLEXD_0 | LIN Transmit Data | 0  |
| PB_00 |           |                 | 736 SIUL2_1      | 0000_0010 | 0x44010DC0 | LIN1_RX               | LINFLEXD_1 | LIN Receive Data  | 1  |
| PB_09 |           |                 |                  | 0000_0001 |            | LIN1_TX               | LINFLEXD_1 | LIN Transmit Data | 0  |
| PB_10 |           |                 | 736 SIUL2_1      | 0000_0011 | 0x44010DC0 | LIN1_RX               | LINFLEXD_1 | LIN Receive Data  | 1  |
| PB_11 |           |                 |                  | 0000_0011 |            | LIN2_TX               | LINFLEXD_2 | LIN Transmit Data | 0  |
| PB_12 |           |                 | 737 SIUL2_1      | 0000_0010 | 0x44010DC4 | LIN2_RX               | LINFLEXD_2 | LIN Receive Data  | 1  |
| PC_04 |           |                 | 736 SIUL2_1      | 0000_0100 | 0x44010DC0 | LIN1_RX               | LINFLEXD_1 | LIN Receive Data  | 1  |
| PC_08 |           |                 |                  | 0000_0010 |            | LIN1_TX               | LINFLEXD_1 | LIN Transmit Data | 0  |
| PC_09 |           |                 |                  | 0000_0001 |            | LIN0_TX               | LINFLEXD_0 | LIN Transmit Data | 0  |
| PC_10 |           |                 | 512 SIUL2_0      | 0000_0010 | 0x4009CA40 | LIN0_RX               | LINFLEXD_0 | LIN Receive Data  | 1  |
| PE_00 |           |                 |                  | 0000_0010 |            | LIN2_TX               | LINFLEXD_2 | LIN Transmit Data | 0  |
| PE_01 |           |                 | 737 SIUL2_1      | 0000_0011 | 0x44010DC4 | LIN2_RX               | LINFLEXD_2 | LIN Receive Data  | 1  |
| PK_11 |           |                 |                  | 0000_0010 |            | LIN2_TX               | LINFLEXD_2 | LIN Transmit Data | 0  |
| PK_12 |           |                 | 737 SIUL2_1      | 0000_0100 | 0x44010DC4 | LIN2_RX               | LINFLEXD_2 | LIN Receive Data  | 1  |
| PK_15 |           |                 |                  | 0000_0010 |            | LIN0_TX               | LINFLEXD_0 | LIN Transmit Data | 0  |
| PL_00 |           |                 | 512 SIUL2_0      | 0000_0100 | 0x4009CA40 | LIN0_RX               | LINFLEXD_0 | LIN Receive Data  | 1  |

Figure 3.2 Uart Physical Unit has selected by UartPhyUnitMapping in EB Tresos

### 3.4 Deviations from Requirements

The driver deviates from the AUTOSAR UART Driver software specification in some places. The table identifies the AUTOSAR requirements that are out of scope, not implemented or not fully implemented for the Uart Driver.

| Term | Definition            |
|------|-----------------------|
| N/S  | Out of scope          |
| N/I  | Not implemented       |
| N/F  | Not fully implemented |

UART is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module. It has vendor-specific requirements and implementation.

## 3.5 Driver Limitations

### 3.5.1 Stop bits limitation

- Driver does not support 3 stop bits for receiver.

**3.5.2 Linflexd\_0 limitation** Linflexd\_0 receiver is used by serial boot process on S32G2, S32G3 and S32R45 platforms, so it affects Linflexd\_0 DMA and Timeout Interrupt features. For avoiding this issue:

- Do not use DMA and Timeout Interrupt features on Linflexd\_0
- Booting with FLEXCAN, QSPI or from external source (Ex: SDcard)

## 3.6 Driver usage and configuration tips

### 3.6.1 How to use User notifications

- The Uart driver can use a function callback in order to notify the application on some of the following events in a transmission: *end transfer*, *rx buffer full*, *tx buffer empty* and *error*. This notification is configured via the configurators supported on the driver: EBT and Design Studio. To enable Uart Callback, on both configurators, GeneralConfiguration/UartCallbackCapability must be set to "True".

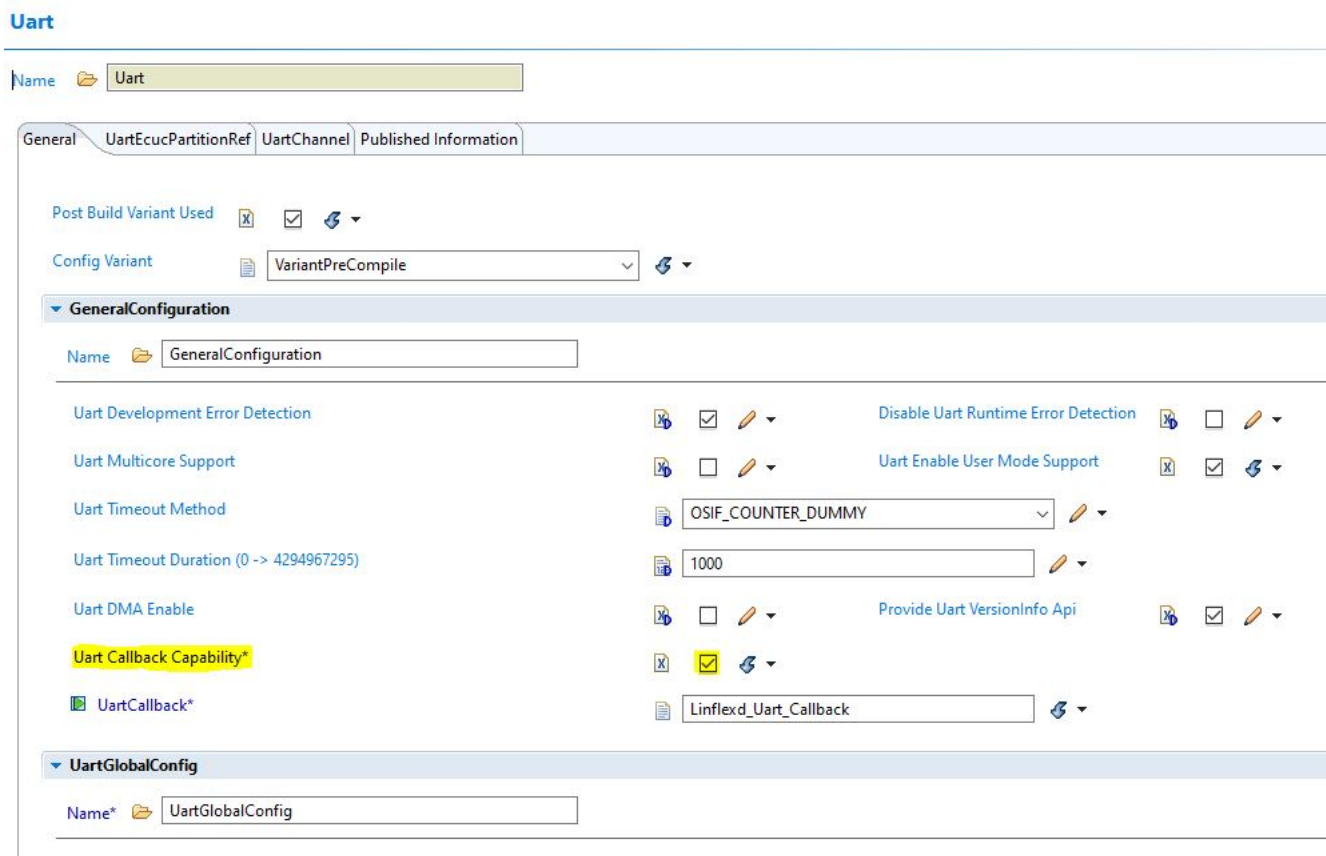


Figure 3.3 Enable Uart Callback Capability

If the HLD driver is used, then the name of the functions must be configured in the following nodes:

## Uart

Name Uart

General UartEcucPartitionRef UartChannel Published Information

Post Build Variant Used ☒

Config Variant VariantPreCompile

**GeneralConfiguration**

Name GeneralConfiguration

|                                         |                                     |                                       |                                     |
|-----------------------------------------|-------------------------------------|---------------------------------------|-------------------------------------|
| Uart Development Error Detection        | <input checked="" type="checkbox"/> | Disable Uart Runtime Error Detection* | <input checked="" type="checkbox"/> |
| Uart Multicore Support*                 | <input checked="" type="checkbox"/> | Uart Enable User Mode Support         | <input checked="" type="checkbox"/> |
| Uart Timeout Method                     | OSIF_COUNTER_DUMMY                  |                                       |                                     |
| Uart Timeout Duration (0 -> 4294967295) | 1000                                |                                       |                                     |
| Uart DMA Enable                         | <input type="checkbox"/>            | Provide Uart VersionInfo Api          | <input checked="" type="checkbox"/> |
| Uart Callback Capability*               | <input checked="" type="checkbox"/> |                                       |                                     |
| UartCallback*                           |                                     | Linflexd_Uart_Callback                |                                     |

**UartGlobalConfig**

Name\* UartGlobalConfig

Figure 3.4 User Notifications

- The signature of the functions shall follow the `Uart_CallbackType` type.
- If the IP driver is used, the function name can be configured in the same path, but the function signature should follow the callback type corresponding to the Hardware Instance used (LinflexD).
- For IP driver, an extra feature can be used: a callback parameter. This variable is a parameter of the notification function and has the `void*` type in order to be a flexible solution for the user. This parameter can be casted to any data type and can be used to pass information from callback call to another. In order to use it in the application, define it in the application as a global variable `void* callback_parameter_name`. Add the variable name (`callback_parameter_name`) in the GeneralConfiguration container of the configurator.
- Another important callback parameter passed is the event that triggered the notification call. Its type depends on the instances of driver in use: `Uart_EventType` or `Ip_name_Uart_Ip_EventType`. Both enums have the following macros:
 

```

_EVENT_RX_FULL = 0x00U,
_EVENT_TX_EMPTY = 0x01U,
_EVENT_END_TRANSFER = 0x02U,
_EVENT_ERROR = 0x03U,

```

Note

The first two can be used with `Uart_SetBuffer`, `Linflexd_Uart_Ip_SetTxBuffer` or `Linflexd_Uart_Ip_SetRxBuffer` API to replace the current buffer when it is full/empty. This approach ensures a continuous transfer.

**3.6.2 How to configure multicore** To enable Multicore Support, GeneralConfiguration/UartMulticore Support must be set to "True".

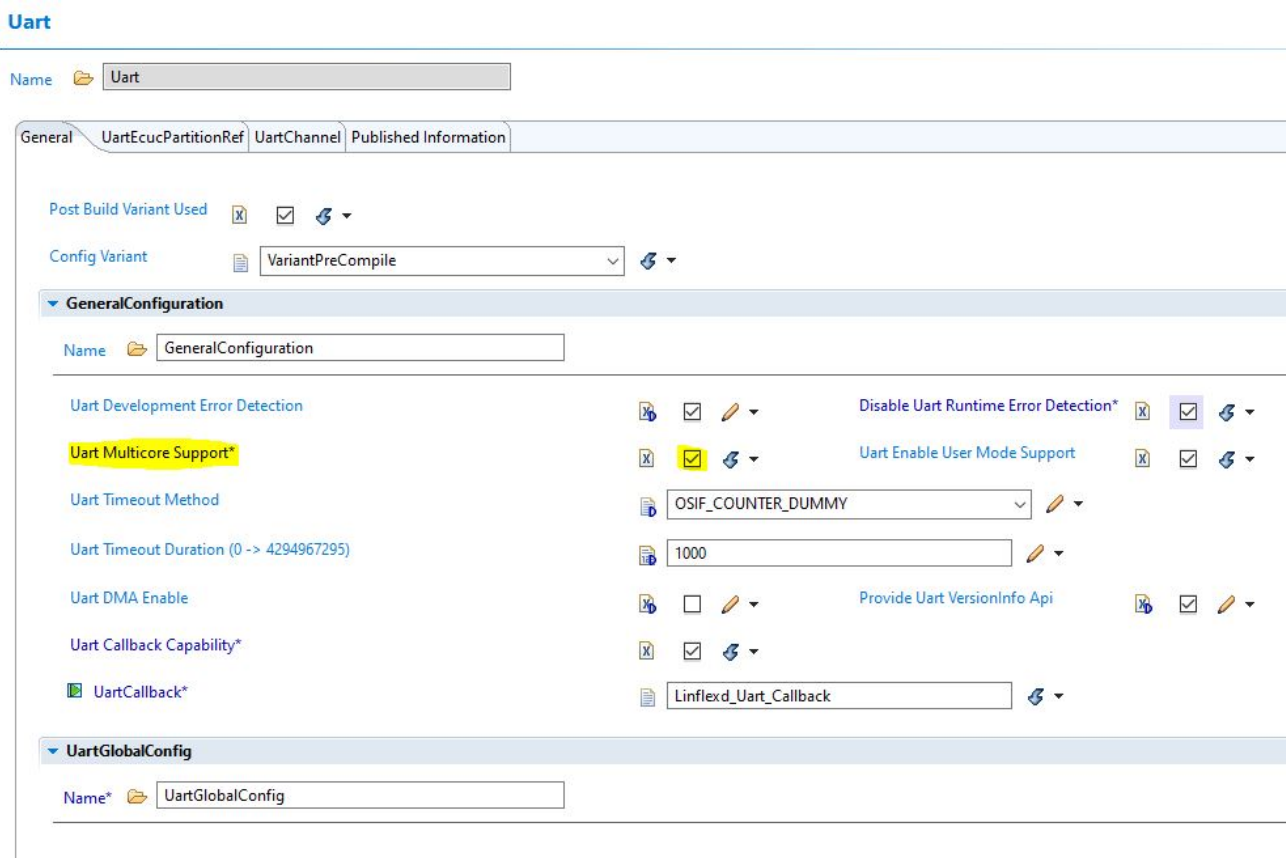


Figure 3.5 Enable Multicore Support

The core for each Uart Channel will be select by UartChannelEcucPartitionRef.

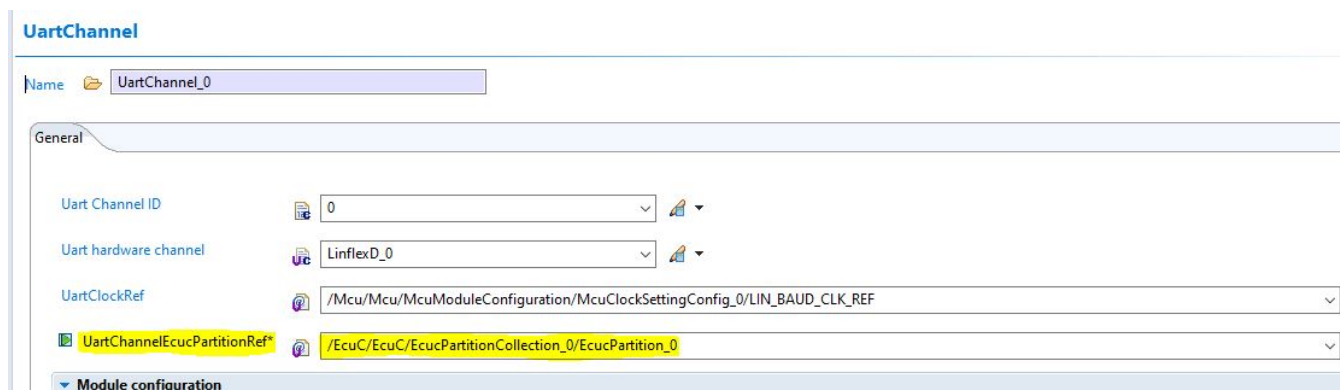


Figure 3.6 Select Uart Channel Ecuc Partition Reference

Each Uart Channel select only one partition (reference from ECUC). Note: All Uart Channels which select UartChannelEcucPartitionRef need to assigned to map with partition in the table below:

| General   UartEcucPartitionRef   UartChannel   Published Information |                                                           |  |
|----------------------------------------------------------------------|-----------------------------------------------------------|--|
| UartEcucPartitionRef                                                 |                                                           |  |
| Index                                                                | UartEcucPartitionRef                                      |  |
| 0                                                                    | /EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_4      |  |
| 1                                                                    | /EcuC/EcuC/EcucPartitionCollection_0/EcucPartition_Cinque |  |

Figure 3.7 Partition for each ExternalDevice

Each partition will refer to a core by OS. A OsApplication will setup for 1 partition mapping with 1 core.

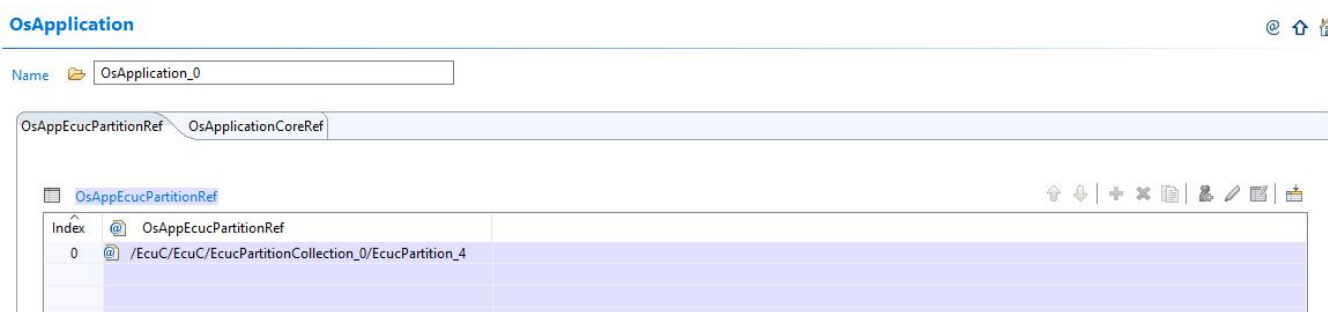


Figure 3.8 Partition is selected by OsApplication\_0



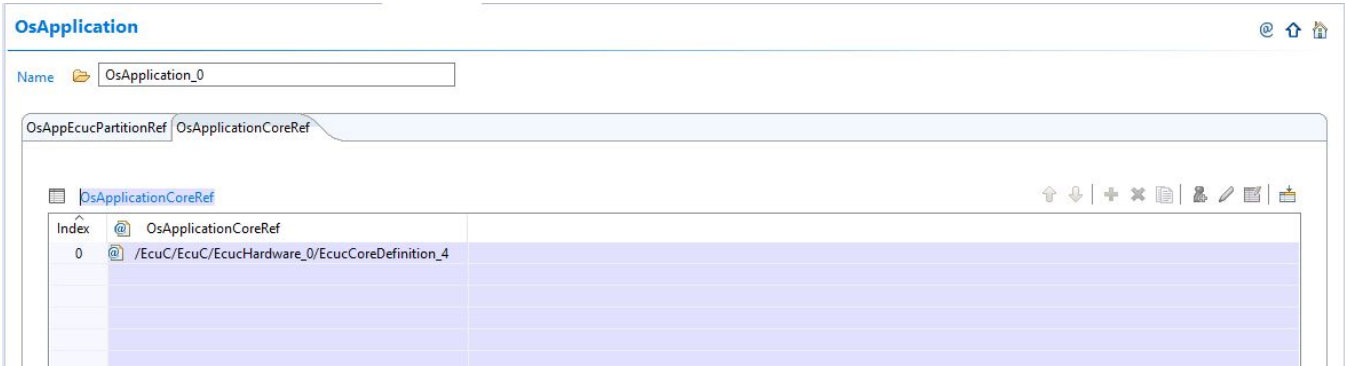


Figure 3.9 Core is also selected by OsApplication\_0

In ECUC, for each EcucCoreDefinition will select a Core.

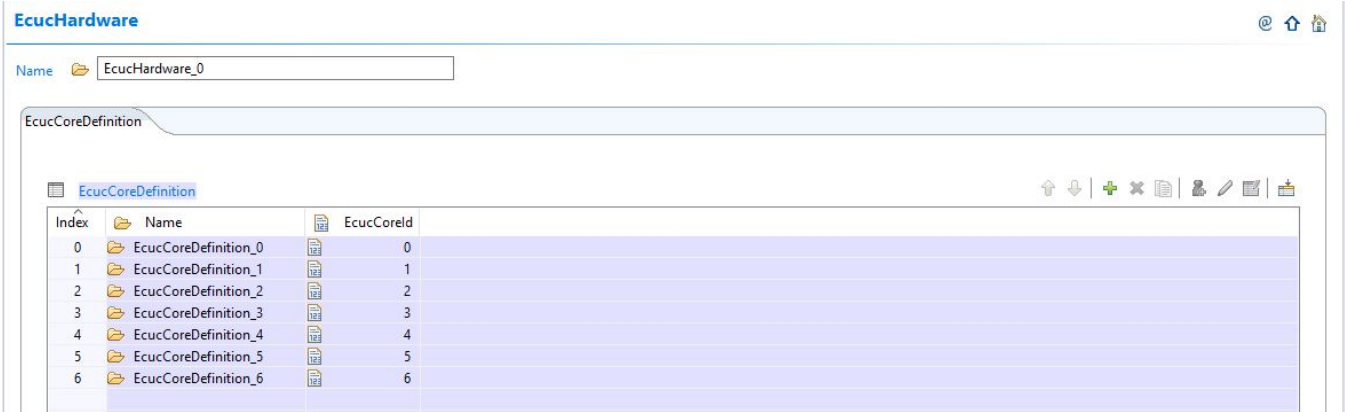
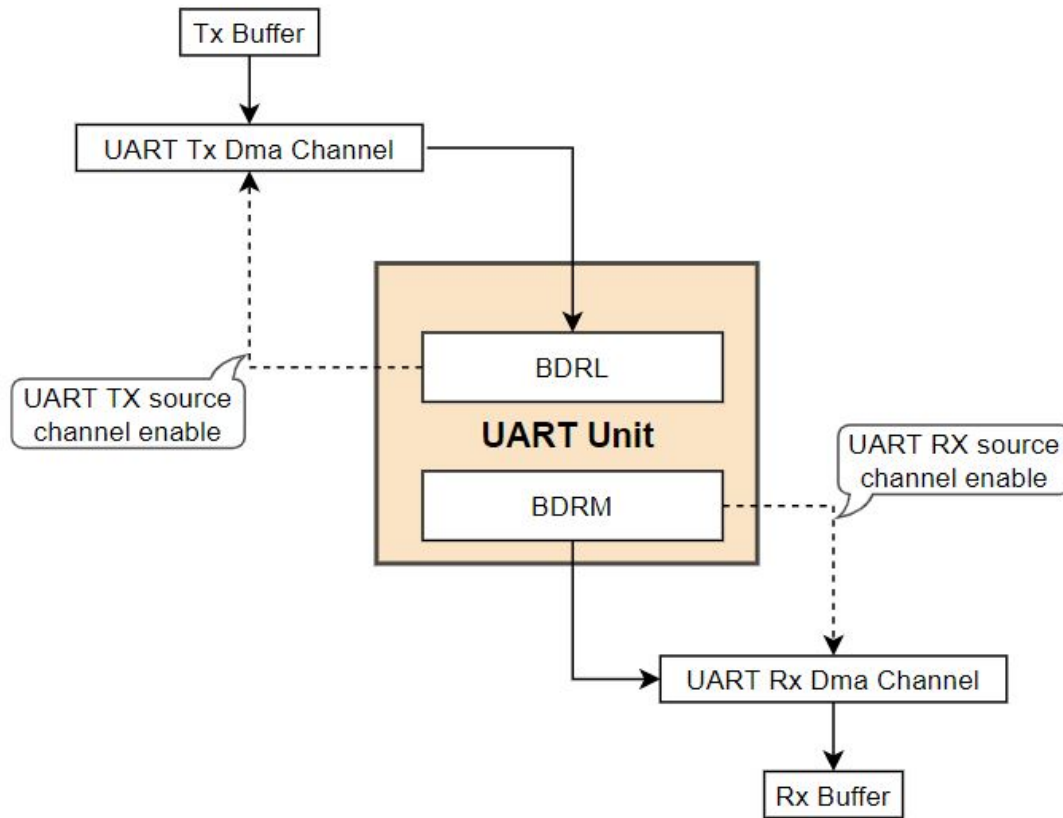


Figure 3.10 Each EcucCoreDefinition will select a Core

**3.6.3 How to configure DMA** This section applies only to Uart units configured for asynchronous transmission and which use DMA for serializing/deserializing data between the hardware unit and the TX/RX buffers (UartInterruptDmaMethod = DMA).



**Figure 3.11 DMA transferring mode internal architecture**

Each Uart unit configured in DMA mode requires 2 distinct DMA channels from the same DMA Mux:

- **Uart TX DMA Channel:** The TX DMA channel used for filling TX FIFO with the data in TX Buffer. This channel is triggered by TX Uart unit event and must be “wired” to “Uart TX source” (configured inside the MCL module – "McIConfig/Dma Logic Channel" container).
- **Uart RX DMA Channel:** The RX DMA channel used for filling RX buffer with the deserialized data. This channel is triggered by RX Uart unit event and must be “wired” to “Uart RX source” (configured inside the MCL module – "McIConfig/Dma Logic Channel" container).

#### Note

- If DMA uses fixed priority arbitration, then the priority must be Uart RX DMA Channel > Uart TX DMA Channel.
- If DMA uses round robin arbitration, no priority constraints are applied on Uart TX DMA Channel, Uart RX DMA Channel priority.

Next figures show an example of DMA configuration for Uart unit.

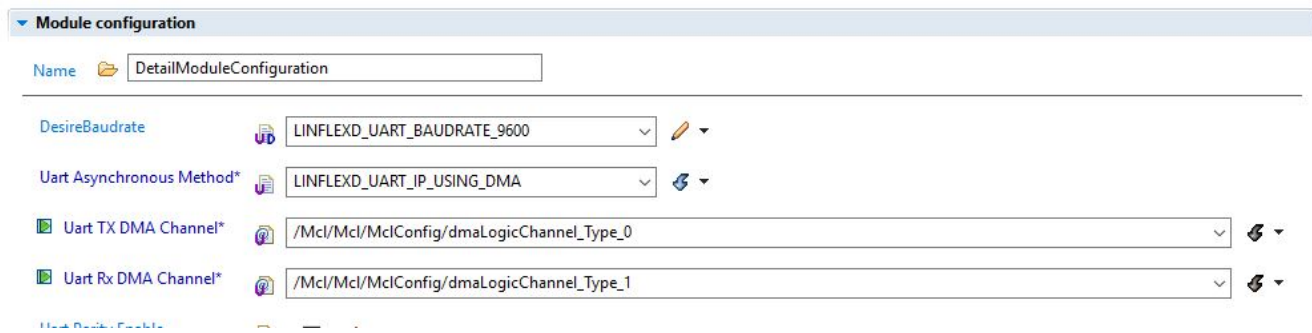


Figure 3.12 DMA Configuration sample for Uart Physical Unit - Uart module in EB Tresos

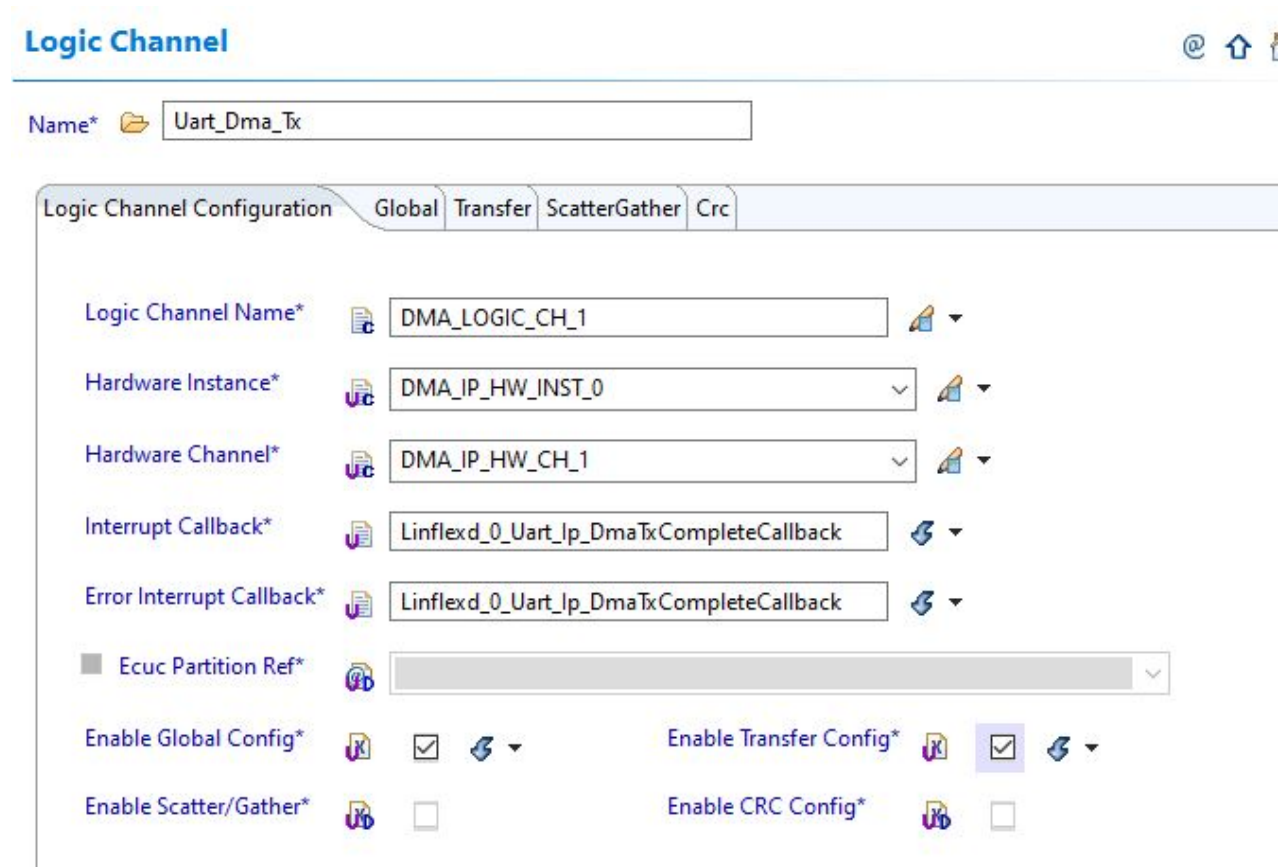


Figure 3.13 DMA TX General configuration - MCL module in EB Tresos

**Logic Channel** @ ↑

Name\*  Uart\_Dma\_Tx

---

Logic Channel Configuration **Global** Transfer ScatterGather Crc

**▼ dmaLogicChannel\_GlobalConfigType**

Name\*  dmaLogicChannel\_GlobalConfigType

---

**▼ Control**

Name\*  dmaLogicChannelConfig\_GlobalControlType

---

Enable Master Id Replication\*  ☐  **Enable Buffered Writes\***  ☐ 

---

**▼ Request**

Name\*  dmaLogicChannelConfig\_GlobalRequestType

---

Enable DMAMUX Trigger\*  ☐  **Enable DMAMUX Source\***  ☒ 

DMAMUX Source\*  DMA\_IP\_REQ\_MUX0\_LINFLEXD0\_TX 

DMAMUX Source\*  DMA\_IP\_REQ\_MUX1\_DISABLED 

Enable DMA Request\*  ☒ 

Figure 3.14 DMA TX Global Configuration - MCL module in EB Tresos

## Logic Channel



Name\* Uart\_Dma\_Rx

| Logic Channel Configuration                  |                                          | Global                  | Transfer                            | ScatterGather | Crc |
|----------------------------------------------|------------------------------------------|-------------------------|-------------------------------------|---------------|-----|
| Logic Channel Name*                          | DMA_LOGIC_CH_0                           |                         |                                     |               |     |
| Hardware Instance*                           | DMA_IP_HW_INST_0                         |                         |                                     |               |     |
| Hardware Channel*                            | DMA_IP_HW_CH_0                           |                         |                                     |               |     |
| Interrupt Callback*                          | Linflexd_0_Uart_Ip_DmaRxCompleteCallback |                         |                                     |               |     |
| Error Interrupt Callback*                    | Linflexd_0_Uart_Ip_DmaRxCompleteCallback |                         |                                     |               |     |
| <input type="checkbox"/> Ecuc Partition Ref* |                                          |                         |                                     |               |     |
| Enable Global Config*                        | <input checked="" type="checkbox"/>      | Enable Transfer Config* | <input checked="" type="checkbox"/> |               |     |
| Enable Scatter/Gather*                       | <input type="checkbox"/>                 | Enable CRC Config*      | <input type="checkbox"/>            |               |     |

Figure 3.15 DMA Rx General configuration - MCL module in EB Tresos

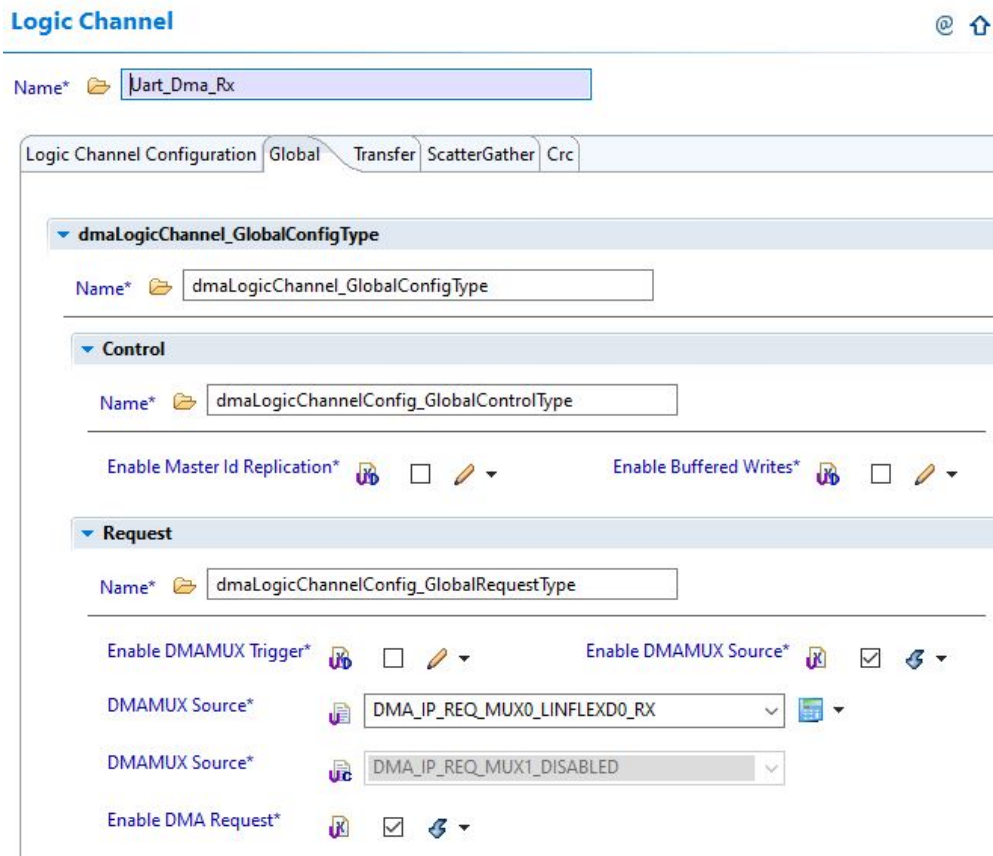


Figure 3.16 DMA Rx Global Configuration - MCL module in EB Tresos

Note:

- About configuration of DMA Callback notification please refer chapter 6.3 Calls to Notification Functions, Callbacks, Callouts in Integration Manual document.
- Address of Buffer is used for Transmit and Receive data with DMA should be placed in UNSPECIFIED\_N←O\_CACHEABLE area please refer chapter 5.6 Data Cache Restrictions in Integration Manual document.

**3.6.4 How to choose the appropriate timeout in general** The timeout need to config in GeneralConfiguration tab of EB Tresos or S32DS tool. For normal operation, this value need to config larger than 1 Uart frame period ( $1s * (\text{number of bit per frame}) / \text{baudrate}$ )

**3.6.5 How to enable Timeout Interrupt to monitor the idle state of the reception line.**

For enabling this mode, user can check the Uart Timeout Enable for each channel that will be configured in this mode.

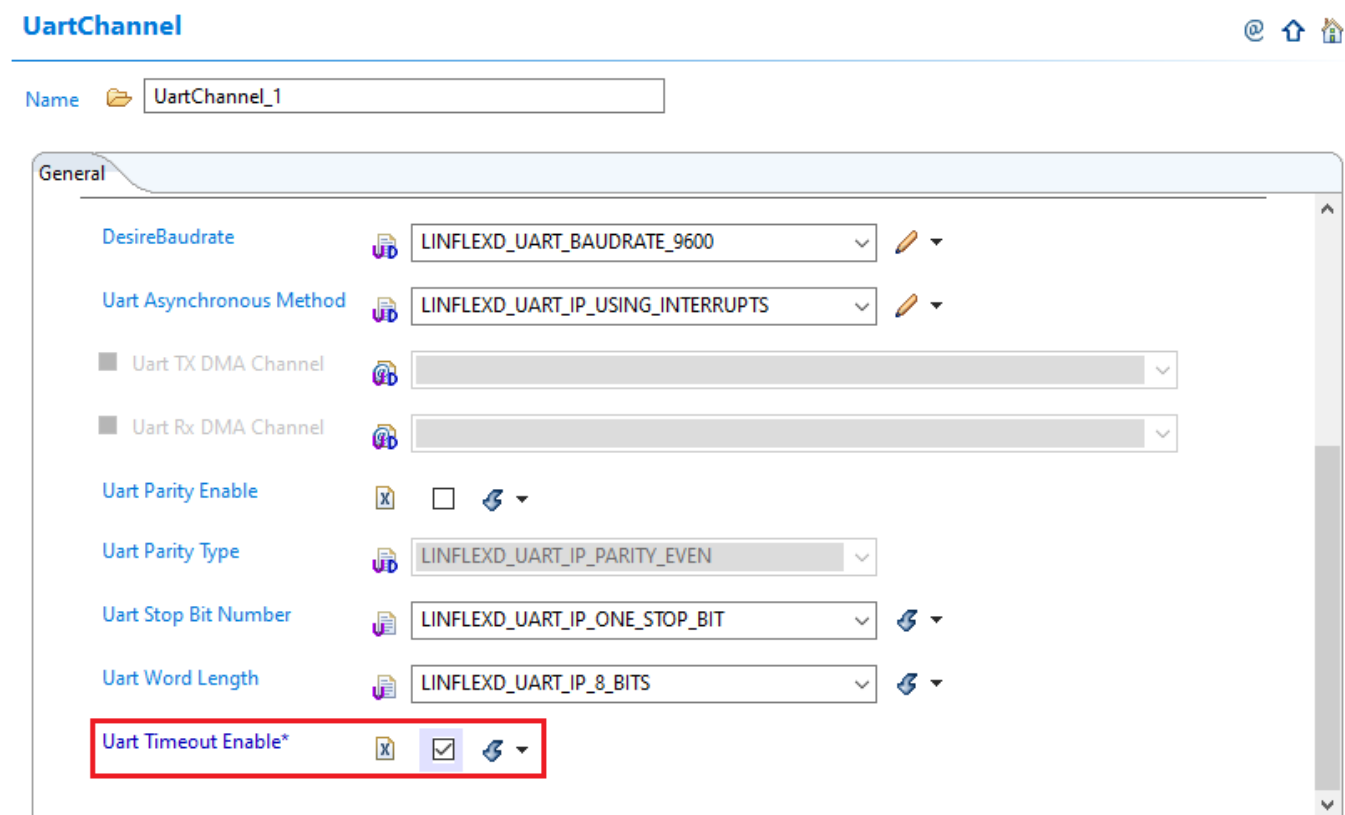


Figure 3.17 Timeout Interrupt configuration

Note: User can use UartCallback function with event LINFLEXD\_UART\_IP\_EVENT\_IDLE\_STATE to handle when Idle state of the reception line is generated.

### 3.7 Runtime errors

The driver generates the following DEM errors at runtime.

| Function         | Error Code           | Condition triggering the error                                                                                                                   |
|------------------|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Uart_SyncSend    | UART_E_TIMEOUT       | If an error occurs in transmit process, then transmitter can not send entire the message for a period of time longer than the configured timeout |
| Uart_SyncReceive | UART_E_TIMEOUT       | If an error occurs in receive process or the receiver waits the message for a period of time longer than the configured timeout                  |
| Uart_Deinit      | UART_E_DEINIT_FAILED | If one or more specific channels are in transmission progress, then Deinit channel will finish unsuccessfully                                    |

### 3.8 Symbolic Names Disclaimer

All containers having `symbolicNameValue` set to `TRUE` in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).



## Chapter 4

### Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Uart](#)
  - Container [GeneralConfiguration](#)
    - \* Parameter [UartDevErrorDetect](#)
    - \* Parameter [DisableUartRuntimeErrorDetect](#)
    - \* Parameter [UartMulticoreSupport](#)
    - \* Parameter [UartEnableUserModeSupport](#)
    - \* Parameter [UartTimeoutMethod](#)
    - \* Parameter [UartTimeoutDuration](#)
    - \* Parameter [UartDmaEnable](#)
    - \* Parameter [UartVersionInfoApi](#)
    - \* Parameter [UartCallbackCapability](#)
    - \* Parameter [UartCallback](#)
    - \* Reference [UartEcucPartitionRef](#)
  - Container [UartGlobalConfig](#)
    - \* Container [UartChannel](#)
      - Parameter [UartChannelId](#)
      - Parameter [UartHwChannel](#)
      - Reference [UartClockRef](#)
      - Reference [UartChannelEcucPartitionRef](#)
      - Container [DetailModuleConfiguration](#)
      - Parameter [DesireBaudrate](#)
      - Parameter [UartInteruptDmaMethod](#)
      - Parameter [UartParityEnable](#)
      - Parameter [UartParityType](#)
      - Parameter [UartStopBitNumber](#)
      - Parameter [UartWordLength](#)
      - Parameter [UartTimeoutEnable](#)
      - Reference [UartDmaTxChannelRef](#)
      - Reference [UartDmaRxChannelRef](#)
  - Container [CommonPublishedInformation](#)

- \* Parameter [ArReleaseMajorVersion](#)
- \* Parameter [ArReleaseMinorVersion](#)
- \* Parameter [ArReleaseRevisionVersion](#)
- \* Parameter [ModuleId](#)
- \* Parameter [SwMajorVersion](#)
- \* Parameter [SwMinorVersion](#)
- \* Parameter [SwPatchVersion](#)
- \* Parameter [VendorApiInfix](#)
- \* Parameter [VendorId](#)

## 4.1 Module Uart

Configuration of the Universal asynchronous receiver-transmitter (Uart) module.

Included containers:

- [GeneralConfiguration](#)
- [UartGlobalConfig](#)
- [CommonPublishedInformation](#)

| Property                | Value                                   |
|-------------------------|-----------------------------------------|
| type                    | ECUC-MODULE-DEF                         |
| lowerMultiplicity       | 1                                       |
| upperMultiplicity       | 1                                       |
| postBuildVariantSupport | true                                    |
| supportedConfigVariants | VARIANT-POST-BUILD, VARIANT-PRE-COMPILE |

## 4.2 Container GeneralConfiguration

GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar Uart driver.

Note: Implementation Specific Parameter.

Included subcontainers:

- None

| Property                     | Value                         |
|------------------------------|-------------------------------|
| type                         | ECUC-PARAM-CONF-CONTAINER-DEF |
| lowerMultiplicity            | 1                             |
| upperMultiplicity            | 1                             |
| postBuildVariantMultiplicity | N/A                           |
| multiplicityConfigClasses    | N/A                           |

### 4.3 Parameter UartDevErrorDetect

UartDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

| Property                     | Value                                                               |
|------------------------------|---------------------------------------------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF                                              |
| origin                       | NXP                                                                 |
| symbolicNameValue            | false                                                               |
| lowerMultiplicity            | 1                                                                   |
| upperMultiplicity            | 1                                                                   |
| postBuildVariantMultiplicity | N/A                                                                 |
| multiplicityConfigClasses    | N/A                                                                 |
| postBuildVariantValue        | false                                                               |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE<br>VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | true                                                                |

### 4.4 Parameter DisableUartRuntimeErrorDetect

DisableUartRuntimeErrorDetect

Switches the Runtime Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

| Property          | Value                  |
|-------------------|------------------------|
| type              | ECUC-BOOLEAN-PARAM-DEF |
| origin            | NXP                    |
| symbolicNameValue | false                  |
| lowerMultiplicity | 1                      |

| Property                     | Value                            |
|------------------------------|----------------------------------|
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| defaultValue                 | false                            |

## 4.5 Parameter UartMulticoreSupport

UartMulticoreEnable

When this parameter is enabled, multi-core feature will be used in Uart driver.

That means mapping the Uart driver to multiple ECUC partitions to make the module API available in this partition.

The Uart driver will operate as an independent instance in each of the partitions.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE |
|                              | VARIANT-POST-BUILD: PRE-COMPILE  |
| defaultValue                 | false                            |

## 4.6 Parameter UartEnableUserModeSupport

When this parameter is enabled, the MDL module will adapt to run from User Mode, with the following measures:

- configuring REG\_PROT for ABC1, ABC2 IPs so that the registers under protection can be accessed from user mode by setting UAA bit in REG\_PROT\_GCR to 1
- using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.
- other module specific measures

for more information, please see chapter 5.7 User Mode Support in IM

Note: Implementation Specific Parameter.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | false                            |

## 4.7 Parameter UartTimeoutMethod

This parameter is used to select between different OsIf counter implementations. For additional details, please refer to the OsIf documentation.

| Property                     | Value                                                                |
|------------------------------|----------------------------------------------------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF                                           |
| origin                       | NXP                                                                  |
| symbolicNameValue            | false                                                                |
| lowerMultiplicity            | 1                                                                    |
| upperMultiplicity            | 1                                                                    |
| postBuildVariantMultiplicity | N/A                                                                  |
| multiplicityConfigClasses    | N/A                                                                  |
| postBuildVariantValue        | false                                                                |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE                                      |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                                     |
| default Value                | OSIF_COUNTER_DUMMY                                                   |
| literals                     | ['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM'] |

## 4.8 Parameter UartTimeoutDuration

Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops. If LinTimeoutMethod is OSIF\_COUNTER\_SYSTEM or OSIF\_COUNTER\_CUSTOM, UartTimeoutDuration is in microsecond value. If LinTimeoutMethod is OSIF\_COUNTER\_DUMMY, the UartTimeoutDuration is number of wait loop.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF           |
| origin                       | AUTOSAR_ECUC                     |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | 1000                             |
| max                          | 4294967295                       |
| min                          | 0                                |

## 4.9 Parameter UartDmaEnable

UartDmaEnable

Check this in order to be able to use DMA in the Uart driver.

Leaving this unchecked will allow the Uart driver to compile with no dependencies from the Mcl driver.

Note: Implementation Specific Parameter.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | false                            |

## 4.10 Parameter UartVersionInfoApi

UartVersionInfoApi

Switches the Uart\_GetVersionInfo function ON or OFF.

Note: Implementation Specific Parameter.

| Property                     | Value                                                               |
|------------------------------|---------------------------------------------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF                                              |
| origin                       | NXP                                                                 |
| symbolicNameValue            | false                                                               |
| lowerMultiplicity            | 1                                                                   |
| upperMultiplicity            | 1                                                                   |
| postBuildVariantMultiplicity | N/A                                                                 |
| multiplicityConfigClasses    | N/A                                                                 |
| postBuildVariantValue        | false                                                               |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE<br>VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | true                                                                |

### 4.11 Parameter UartCallbackCapability

When this parameter is enabled, Callback feature will be used in Uart driver, with the following functions:

- a) completion of a reception or sending operation.
- b) reporting communication errors.

| Property                     | Value                                                               |
|------------------------------|---------------------------------------------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF                                              |
| origin                       | NXP                                                                 |
| symbolicNameValue            | false                                                               |
| lowerMultiplicity            | 1                                                                   |
| upperMultiplicity            | 1                                                                   |
| postBuildVariantMultiplicity | N/A                                                                 |
| multiplicityConfigClasses    | N/A                                                                 |
| postBuildVariantValue        | false                                                               |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE<br>VARIANT-PRE-COMPILE: PRE-COMPILE |
| default Value                | false                                                               |

### 4.12 Parameter UartCallback

Callback function will be used in Uart driver.

Note: Implementation Specific Parameter.



| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-FUNCTION-NAME-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 0                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | false                            |
| multiplicityConfigClasses    | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: PRE-COMPILE  |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| defaultValue                 | NULL_PTR                         |

### 4.13 Reference UartEcucPartitionRef

ECUC\_Uart\_00244.Maps the Uart driver to zero or multiple ECUC partitions to make the driver

API available in the according partition.

| Property                     | Value                                                        |
|------------------------------|--------------------------------------------------------------|
| type                         | ECUC-REFERENCE-DEF                                           |
| origin                       | AUTOSAR_ECUC                                                 |
| lowerMultiplicity            | 0                                                            |
| upperMultiplicity            | Infinite                                                     |
| postBuildVariantMultiplicity | true                                                         |
| multiplicityConfigClasses    | VARIANT-PRE-COMPILE: PRE-COMPILE                             |
|                              | VARIANT-POST-BUILD: PRE-COMPILE                              |
| postBuildVariantValue        | true                                                         |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE                             |
|                              | VARIANT-POST-BUILD: PRE-COMPILE                              |
| requiresSymbolicNameValue    | False                                                        |
| destination                  | /AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition |

### 4.14 Container UartGlobalConfig

This container contains the global configuration parameter of the Uart driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exit once per configuration set.

Included subcontainers:

- [UartChannel](#)

| Property                     | Value                         |
|------------------------------|-------------------------------|
| type                         | ECUC-PARAM-CONF-CONTAINER-DEF |
| lowerMultiplicity            | 1                             |
| upperMultiplicity            | 1                             |
| postBuildVariantMultiplicity | N/A                           |
| multiplicityConfigClasses    | N/A                           |

## 4.15 Container UartChannel

This container contains the configuration (parameters) of the Uart Controller(s).

Note: "User should use unique names for naming the Uart channels across different UartGlobalConfig Sets."

Included subcontainers:

- [DetailModuleConfiguration](#)

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-PARAM-CONF-CONTAINER-DEF    |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | Infinite                         |
| postBuildVariantMultiplicity | false                            |
| multiplicityConfigClasses    | VARIANT-PRE-COMPILE: PRE-COMPILE |
|                              | VARIANT-POST-BUILD: PRE-COMPILE  |

## 4.16 Parameter UartChannelId

Identifies the Uart channel.

Note: Implementation Specific Parameter.

| Property                     | Value                  |
|------------------------------|------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF |
| origin                       | NXP                    |
| symbolicNameValue            | true                   |
| lowerMultiplicity            | 1                      |
| upperMultiplicity            | 1                      |
| postBuildVariantMultiplicity | N/A                    |
| multiplicityConfigClasses    | N/A                    |
| postBuildVariantValue        | false                  |

| Property           | Value                            |
|--------------------|----------------------------------|
| valueConfigClasses | VARIANT-POST-BUILD: PRE-COMPILE  |
|                    | VARIANT-PRE-COMPILE: PRE-COMPILE |
| defaultValue       | 1                                |
| max                | 3                                |
| min                | 0                                |

## 4.17 Parameter UartHwChannel

Selects the physical Uart Channel.

Note: Implementation Specific Parameter.

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF                 |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | true                                       |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE           |
|                              | VARIANT-POST-BUILD: POST-BUILD             |
| defaultValue                 | LinflexD_0                                 |
| literals                     | ['LinflexD_0', 'LinflexD_1', 'LinflexD_2'] |

## 4.18 Reference UartClockRef

Reference to the Uart clock source configuration, which is set into the MCU driver configuration.

This clock source is used for configure Uart baudrate.

| Property                     | Value              |
|------------------------------|--------------------|
| type                         | ECUC-REFERENCE-DEF |
| origin                       | NXP                |
| lowerMultiplicity            | 1                  |
| upperMultiplicity            | 1                  |
| postBuildVariantMultiplicity | N/A                |
| multiplicityConfigClasses    | N/A                |
| postBuildVariantValue        | true               |

| Property                  | Value                                                                                         |
|---------------------------|-----------------------------------------------------------------------------------------------|
| valueConfigClasses        | VARIANT-PRE-COMPILE: PRE-COMPILE                                                              |
|                           | VARIANT-POST-BUILD: POST-BUILD                                                                |
| requiresSymbolicNameValue | False                                                                                         |
| destination               | /AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔<br>Config/McuClockReferencePoint |

## 4.19 Reference UartChannelEcucPartitionRef

Maps one single Uart channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions where the Uart driver is mapped to.

| Property                     | Value                                                       |
|------------------------------|-------------------------------------------------------------|
| type                         | ECUC-REFERENCE-DEF                                          |
| origin                       | AUTOSAR_ECUC                                                |
| lowerMultiplicity            | 0                                                           |
| upperMultiplicity            | 1                                                           |
| postBuildVariantMultiplicity | true                                                        |
| multiplicityConfigClasses    | VARIANT-PRE-COMPILE: PRE-COMPILE                            |
|                              | VARIANT-POST-BUILD: PRE-COMPILE                             |
| postBuildVariantValue        | true                                                        |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE                            |
|                              | VARIANT-POST-BUILD: PRE-COMPILE                             |
| requiresSymbolicNameValue    | False                                                       |
| destination                  | /AUTOSAR/EcuDefs/EcuC/EcucPartitionCollection/EcucPartition |

## 4.20 Container DetailModuleConfiguration

This container contains the configuration (parameters) of the Module Configuration.

Included subcontainers:

- None

| Property                     | Value                         |
|------------------------------|-------------------------------|
| type                         | ECUC-PARAM-CONF-CONTAINER-DEF |
| lowerMultiplicity            | 1                             |
| upperMultiplicity            | 1                             |
| postBuildVariantMultiplicity | N/A                           |
| multiplicityConfigClasses    | N/A                           |

## 4.21 Parameter DesireBaudrate

This parameter is user's desire baudrate.

Note: Implementation Specific Parameter.

| Property                     | Value                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| origin                       | AUTOSAR_ECUC                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| symbolicNameValue            | false                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| lowerMultiplicity            | 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| upperMultiplicity            | 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| postBuildVariantMultiplicity | N/A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| multiplicityConfigClasses    | N/A                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| postBuildVariantValue        | true                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|                              | VARIANT-POST-BUILD: POST-BUILD                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| defaultValue                 | LINFLEXD_UART_BAUDRATE_9600                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| literals                     | ['LINFLEXD_UART_BAUDRATE_1200', 'LINFLEXD_UART_BAUDRATE_2400', 'LINFLEXD_UART_BAUDRATE_4800', 'LINFLEXD_UART_BAUDRATE_7200', 'LINFLEXD_UART_BAUDRATE_9600', 'LINFLEXD_UART_BAUDRATE_14400', 'LINFLEXD_UART_BAUDRATE_19200', 'LINFLEXD_UART_BAUDRATE_28800', 'LINFLEXD_UART_BAUDRATE_38400', 'LINFLEXD_UART_BAUDRATE_57600', 'LINFLEXD_UART_BAUDRATE_115200', 'LINFLEXD_UART_BAUDRATE_230400', 'LINFLEXD_UART_BAUDRATE_460800', 'LINFLEXD_UART_BAUDRATE_921600', 'LINFLEXD_UART_BAUDRATE_1843200'] |

## 4.22 Parameter UartInterruptDmaMethod

Configures the mechanism used by the 'AsyncSend' or 'AsyncReceive' function (interrupts or DMA).

Note: Implementation Specific Parameter.

| Property                     | Value                          |
|------------------------------|--------------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF     |
| origin                       | NXP                            |
| symbolicNameValue            | false                          |
| lowerMultiplicity            | 1                              |
| upperMultiplicity            | 1                              |
| postBuildVariantMultiplicity | N/A                            |
| multiplicityConfigClasses    | N/A                            |
| postBuildVariantValue        | true                           |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD |

| Property     | Value                                                                    |
|--------------|--------------------------------------------------------------------------|
|              | VARIANT-PRE-COMPILE: PRE-COMPILE                                         |
| defaultValue | LINFLEXD_UART_IP_USING_INTERRUPTS                                        |
| literals     | ['LINFLEXD_UART_IP_USING_INTERRUPTS', 'LINFLEXD_UART_IP←<br>_USING_DMA'] |

## 4.23 Parameter UartParityEnable

UartParityEnable

Check this in order to be able to use parity for UART bytes.

Note: Implementation Specific Parameter.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD   |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE |
| defaultValue                 | false                            |

## 4.24 Parameter UartParityType

Configures the type of parity to be used for UART bytes.

Note: Implementation Specific Parameter.

| Property                     | Value                      |
|------------------------------|----------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF |
| origin                       | NXP                        |
| symbolicNameValue            | false                      |
| lowerMultiplicity            | 1                          |
| upperMultiplicity            | 1                          |
| postBuildVariantMultiplicity | N/A                        |
| multiplicityConfigClasses    | N/A                        |

| Property              | Value                                                                                                                          |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| postBuildVariantValue | true                                                                                                                           |
| valueConfigClasses    | VARIANT-POST-BUILD: POST-BUILD                                                                                                 |
|                       | VARIANT-PRE-COMPILE: PRE-COMPILE                                                                                               |
| defaultValue          | LINFLEXD_UART_IP_PARITY_EVEN                                                                                                   |
| literals              | ['LINFLEXD_UART_IP_PARITY_EVEN', 'LINFLEXD_UART_IP_PARITY_ODD', 'LINFLEXD_UART_IP_PARITY_ZERO', 'LINFLEXD_UART_IP_PARITY_ONE'] |

## 4.25 Parameter UartStopBitNumber

Configures the number of stop bit to be used for UART frame.

Note: Implementation Specific Parameter.

| Property                     | Value                                                              |
|------------------------------|--------------------------------------------------------------------|
| type                         | ECUC-ENUMERATION-PARAM-DEF                                         |
| origin                       | NXP                                                                |
| symbolicNameValue            | false                                                              |
| lowerMultiplicity            | 1                                                                  |
| upperMultiplicity            | 1                                                                  |
| postBuildVariantMultiplicity | N/A                                                                |
| multiplicityConfigClasses    | N/A                                                                |
| postBuildVariantValue        | true                                                               |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD                                     |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                                   |
| defaultValue                 | LINFLEXD_UART_IP_ONE_STOP_BIT                                      |
| literals                     | ['LINFLEXD_UART_IP_ONE_STOP_BIT', 'LINFLEXD_UART_IP_TWO_STOP_BIT'] |

## 4.26 Parameter UartWordLength

Configures the word length in UART mode.

Note: Implementation Specific Parameter.

| Property          | Value                      |
|-------------------|----------------------------|
| type              | ECUC-ENUMERATION-PARAM-DEF |
| origin            | NXP                        |
| symbolicNameValue | false                      |
| lowerMultiplicity | 1                          |

| Property                     | Value                                                                                                          |
|------------------------------|----------------------------------------------------------------------------------------------------------------|
| upperMultiplicity            | 1                                                                                                              |
| postBuildVariantMultiplicity | N/A                                                                                                            |
| multiplicityConfigClasses    | N/A                                                                                                            |
| postBuildVariantValue        | true                                                                                                           |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD                                                                                 |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                                                                               |
| defaultValue                 | LINFLEXD_UART_IP_8_BITS                                                                                        |
| literals                     | ['LINFLEXD_UART_IP_7_BITS', 'LINFLEXD_UART_IP_8_BITS', 'LINFLEXD_UART_IP_15_BITS', 'LINFLEXD_UART_IP_16_BITS'] |

## 4.27 Parameter UartTimeoutEnable

UartTimeoutEnable

This checks enables the Uart Timeout Interrupt for the current channel.

| Property                     | Value                            |
|------------------------------|----------------------------------|
| type                         | ECUC-BOOLEAN-PARAM-DEF           |
| origin                       | NXP                              |
| symbolicNameValue            | false                            |
| lowerMultiplicity            | 1                                |
| upperMultiplicity            | 1                                |
| postBuildVariantMultiplicity | N/A                              |
| multiplicityConfigClasses    | N/A                              |
| postBuildVariantValue        | false                            |
| valueConfigClasses           | VARIANT-PRE-COMPILE: PRE-COMPILE |
|                              | VARIANT-POST-BUILD: PRE-COMPILE  |
| defaultValue                 | false                            |

## 4.28 Reference UartDmaTxChannelRef

Reference to the DMA TX channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

| Property          | Value              |
|-------------------|--------------------|
| type              | ECUC-REFERENCE-DEF |
| origin            | NXP                |
| lowerMultiplicity | 0                  |
| upperMultiplicity | 1                  |



| Property                     | Value                                                |
|------------------------------|------------------------------------------------------|
| postBuildVariantMultiplicity | false                                                |
| multiplicityConfigClasses    | VARIANT-POST-BUILD: POST-BUILD                       |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                     |
| postBuildVariantValue        | true                                                 |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD                       |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                     |
| requiresSymbolicNameValue    | False                                                |
| destination                  | /AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type |

## 4.29 Reference UartDmaRxChannelRef

Reference to the DMA Rx channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

| Property                     | Value                                                |
|------------------------------|------------------------------------------------------|
| type                         | ECUC-REFERENCE-DEF                                   |
| origin                       | NXP                                                  |
| lowerMultiplicity            | 0                                                    |
| upperMultiplicity            | 1                                                    |
| postBuildVariantMultiplicity | false                                                |
| multiplicityConfigClasses    | VARIANT-POST-BUILD: POST-BUILD                       |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                     |
| postBuildVariantValue        | true                                                 |
| valueConfigClasses           | VARIANT-POST-BUILD: POST-BUILD                       |
|                              | VARIANT-PRE-COMPILE: PRE-COMPILE                     |
| requiresSymbolicNameValue    | False                                                |
| destination                  | /AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type |

## 4.30 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

| Property                     | Value                         |
|------------------------------|-------------------------------|
| type                         | ECUC-PARAM-CONF-CONTAINER-DEF |
| lowerMultiplicity            | 1                             |
| upperMultiplicity            | 1                             |
| postBuildVariantMultiplicity | N/A                           |
| multiplicityConfigClasses    | N/A                           |

### 4.31 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                     |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 | 4                                          |
| max                          | 4                                          |
| min                          | 4                                          |

### 4.32 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

| Property                     | Value                                     |
|------------------------------|-------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                    |
| origin                       | NXP                                       |
| symbolicNameValue            | false                                     |
| lowerMultiplicity            | 1                                         |
| upperMultiplicity            | 1                                         |
| postBuildVariantMultiplicity | N/A                                       |
| multiplicityConfigClasses    | N/A                                       |
| postBuildVariantValue        | false                                     |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION |

| Property     | Value                                      |
|--------------|--------------------------------------------|
|              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue | 4                                          |
| max          | 4                                          |
| min          | 4                                          |

### 4.33 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                     |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 | 0                                          |
| max                          | 0                                          |
| min                          | 0                                          |

### 4.34 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

| Property                     | Value                                     |
|------------------------------|-------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                    |
| origin                       | NXP                                       |
| symbolicNameValue            | false                                     |
| lowerMultiplicity            | 1                                         |
| upperMultiplicity            | 1                                         |
| postBuildVariantMultiplicity | N/A                                       |
| multiplicityConfigClasses    | N/A                                       |
| postBuildVariantValue        | false                                     |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION |

| Property     | Value                                      |
|--------------|--------------------------------------------|
|              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue | 255                                        |
| max          | 255                                        |
| min          | 255                                        |

### 4.35 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                     |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 | 4                                          |
| max                          | 4                                          |
| min                          | 4                                          |

### 4.36 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

| Property                     | Value                  |
|------------------------------|------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF |
| origin                       | NXP                    |
| symbolicNameValue            | false                  |
| lowerMultiplicity            | 1                      |
| upperMultiplicity            | 1                      |
| postBuildVariantMultiplicity | N/A                    |
| multiplicityConfigClasses    | N/A                    |

| Property              | Value                                      |
|-----------------------|--------------------------------------------|
| postBuildVariantValue | false                                      |
| valueConfigClasses    | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                       | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue          | 0                                          |
| max                   | 0                                          |
| min                   | 0                                          |

### 4.37 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                     |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 | 2                                          |
| max                          | 2                                          |
| min                          | 2                                          |

### 4.38 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>\_\_>VendorId>\_\_<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name

Can\_Write defined in the SWS will translate to Can\_123\_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-STRING-PARAM-DEF                      |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 |                                            |

## 4.39 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

| Property                     | Value                                      |
|------------------------------|--------------------------------------------|
| type                         | ECUC-INTEGER-PARAM-DEF                     |
| origin                       | NXP                                        |
| symbolicNameValue            | false                                      |
| lowerMultiplicity            | 1                                          |
| upperMultiplicity            | 1                                          |
| postBuildVariantMultiplicity | N/A                                        |
| multiplicityConfigClasses    | N/A                                        |
| postBuildVariantValue        | false                                      |
| valueConfigClasses           | VARIANT-POST-BUILD: PUBLISHED-INFORMATION  |
|                              | VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION |
| defaultValue                 | 43                                         |
| max                          | 43                                         |
| min                          | 43                                         |



# Chapter 5

## Module Index

### 5.1 Software Specification

Here is a list of all modules:

|                             |    |
|-----------------------------|----|
| Linflexd UART IPL . . . . . | 46 |
| UART Driver . . . . .       | 60 |



## Chapter 6

### Module Documentation

#### 6.1 Linflexd UART IPL

##### 6.1.1 Detailed Description

##### Data Structures

- struct [Linflexd\\_Uart\\_Ip\\_StateStructureType](#)  
*Runtime State of the UART driver. [More...](#)*
- struct [Linflexd\\_Uart\\_Ip\\_UserConfigType](#)  
*LINFLEXD configuration structure. [More...](#)*

##### Types Reference

- typedef void(\* [Linflexd\\_Uart\\_Ip\\_CallbackType](#)) (const uint8 HwInstance, const [Linflexd\\_Uart\\_Ip\\_EventType](#) Event, void \*UserData)  
*Callback for all peripherals which support UART features.*

##### Enum Reference

- enum [Linflexd\\_Uart\\_Ip\\_TransferType](#)  
*Type of UART a-sync transfer (based on interrupts or DMA).*
- enum [Linflexd\\_Uart\\_Ip\\_DataDirectionType](#)  
*The type operation of an LinflexD Uart channel.*
- enum [Linflexd\\_Uart\\_Ip\\_StatusType](#)  
*Driver status type.*
- enum [Linflexd\\_Uart\\_Ip\\_EventType](#)  
*Define the enum of the Events which can trigger UART callback.*
- enum [Linflexd\\_Uart\\_Ip\\_BaudrateType](#)  
*Baudrate values supported by the driver.*

## Function Reference

- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_SetBaudrate](#) (const uint8 Instance, const [Linflexd\\_Uart\\_Ip\\_BaudrateType](#) DesiredBaudRate, const uint32 ClockFrequency)  
*Sets the baud rate for UART communication.*
- void [Linflexd\\_Uart\\_Ip\\_GetBaudrate](#) (const uint8 Instance, uint32 \*ConfiguredBaudRate)  
*Gets the baud rate for UART communication.*
- void [Linflexd\\_Uart\\_Ip\\_Init](#) (const uint8 Instance, const [Linflexd\\_Uart\\_Ip\\_UserConfigType](#) \*UserConfig)  
*Initializes a LINFLEXD instance for UART operations.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_Deinit](#) (const uint8 Instance)  
*Shuts down the UART functionality of the LINFLEXD module by disabling interrupts and transmitter/receiver.*
- void [Linflexd\\_Uart\\_Ip\\_SetTxBuffer](#) (const uint8 Instance, const uint8 \*TxBuff, const uint32 TxSize)  
*Sets the internal driver reference to the tx buffer.*
- void [Linflexd\\_Uart\\_Ip\\_SetRxBuffer](#) (const uint8 Instance, uint8 \*RxBuff, const uint32 RxSize)  
*Sets the internal driver reference to the rx buffer.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_AbortReceivingData](#) (const uint8 Instance)  
*Terminates a non-blocking receive early.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_AbortSendingData](#) (const uint8 Instance)  
*Terminates a non-blocking transfer(send) early.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_SyncSend](#) (const uint8 Instance, const uint8 \*TxBuff, const uint32 TxSize, const uint32 Timeout)  
*Sends data using LINFLEXD module in UART mode with polling method.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_SyncReceive](#) (const uint8 Instance, uint8 \*RxBuff, const uint32 RxSize, const uint32 Timeout)  
*Sends data using LINFLEXD module in UART mode with polling method.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_AsyncReceive](#) (const uint8 Instance, uint8 \*RxBuff, const uint32 RxSize)  
*Starts data reception from the LINFLEXD module in UART mode with non-blocking method.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_AsyncSend](#) (const uint8 Instance, const uint8 \*TxBuff, const uint32 TxSize)  
*Sends data using LINFLEXD module in UART mode with non-blocking method.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_GetTransmitStatus](#) (const uint8 Instance, uint32 \*BytesRemaining)  
*Returns whether the status of the previous transmission.*
- [Linflexd\\_Uart\\_Ip\\_StatusType Linflexd\\_Uart\\_Ip\\_GetReceiveStatus](#) (const uint8 Instance, uint32 \*BytesRemaining)  
*Returns whether the status of the previous reception.*

## 6.1.2 Data Structure Documentation

### 6.1.2.1 struct Linflexd\_Uart\_Ip\_StateStructureType

Runtime State of the UART driver.

Definition at line 175 of file Linflexd\_Uart\_Ip\_Types.h.

## Data Fields

| Type                                                 | Name                | Description                                                                      |
|------------------------------------------------------|---------------------|----------------------------------------------------------------------------------|
| uint32                                               | Baudrate            | Variable that indicates if structure belongs to an instance already initialized. |
| const uint8 *                                        | TxBuff              | The buffer of data being sent.                                                   |
| uint8 *                                              | RxBuff              | The buffer of received data.                                                     |
| volatile uint32                                      | TxSize              | The remaining number of bytes to be transmitted.                                 |
| volatile uint32                                      | RxSize              | The remaining number of bytes to be received.                                    |
| volatile boolean                                     | IsTxBusy            | True if there is an active transmit.                                             |
| volatile boolean                                     | IsRxBusy            | True if there is an active receive.                                              |
| volatile <a href="#">Linflexd_Uart_Ip_StatusType</a> | TransmitStatus      | Status of last driver transmit operation.                                        |
| volatile <a href="#">Linflexd_Uart_Ip_StatusType</a> | ReceiveStatus       | Status of last driver receive operation.                                         |
| boolean                                              | IsDriverInitialized | Variable that indicates if driver is initialized or not.                         |

## 6.1.2.2 struct Linflexd\_Uart\_Ip\_UserConfigType

LINFLEXD configuration structure.

Implements : [Linflexd\\_Uart\\_Ip\\_UserConfigType](#)

Definition at line 195 of file Linflexd\_Uart\_Ip\_Types.h.

## Data Fields

| Type                                                  | Name                      | Description                                                                                                                             |
|-------------------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| uint32                                                | BaudRate                  | Baudrate value.                                                                                                                         |
| uint32                                                | BaudRateMantissa          | Baudrate mantissa.                                                                                                                      |
| uint8                                                 | BaudRateFractionalDivisor | Baudrate Divisor.                                                                                                                       |
| boolean                                               | ParityCheck               | Parity control - enabled/disabled. :<br>This parameter is valid only for 8/16 bits words; for 7/15 word length parity is always enabled |
| <a href="#">Linflexd_Uart_Ip_ParityType</a>           | ParityType                | always 0/always 1/even/odd                                                                                                              |
| <a href="#">Linflexd_Uart_Ip_StopBitsCountType</a>    | StopBitsCount             | number of stop bits, 1 stop bit (default) or 2 stop bits                                                                                |
| <a href="#">Linflexd_Uart_Ip_WordLengthType</a>       | WordLength                | number of bits per transmitted/received word                                                                                            |
| <a href="#">Linflexd_Uart_Ip_TransferType</a>         | TransferType              | Type of UART transfer (interrupt/dma based)                                                                                             |
| <a href="#">Linflexd_Uart_Ip_CallbackType</a>         | Callback                  | Callback to invoke for handling uart event.                                                                                             |
| void *                                                | CallbackParam             | Receive callback parameter pointer.                                                                                                     |
| <a href="#">Linflexd_Uart_Ip_StateStructureType</a> * | StateStruct               | Runtime state structure reference.                                                                                                      |

### 6.1.3 Types Reference

#### 6.1.3.1 Linflexd\_Uart\_Ip\_CallbackType

```
typedef void(* Linflexd_Uart_Ip_CallbackType) (const uint8 HwInstance, const Linflexd_Uart_Ip_EventType
Event, void *UserData)
```

Callback for all peripherals which support UART features.

Definition at line 165 of file Linflexd\_Uart\_Ip\_Types.h.

### 6.1.4 Enum Reference

#### 6.1.4.1 Linflexd\_Uart\_Ip\_TransferType

```
enum Linflexd_Uart_Ip_TransferType
```

Type of UART a-sync transfer (based on interrupts or DMA).

Enumerator

|                                   |                                                          |
|-----------------------------------|----------------------------------------------------------|
| LINFLEXD_UART_IP_USING_DMA        | The driver will use DMA to perform UART transfer.        |
| LINFLEXD_UART_IP_USING_INTERRUPTS | The driver will use interrupts to perform UART transfer. |

Definition at line 101 of file Linflexd\_Uart\_Ip\_Types.h.

#### 6.1.4.2 Linflexd\_Uart\_Ip\_DataDirectionType

```
enum Linflexd_Uart_Ip_DataDirectionType
```

The type operation of an LinflexD Uart channel.

Enumerator

|                          |                          |
|--------------------------|--------------------------|
| LINFLEXD_UART_IP_SEND    | The sending operation.   |
| LINFLEXD_UART_IP_RECEIVE | The receiving operation. |

Definition at line 110 of file Linflexd\_Uart\_Ip\_Types.h.

### 6.1.4.3 Linflexd\_Uart\_Ip\_StatusType

enum `Linflexd_Uart_Ip_StatusType`

Driver status type.

Enumerator

|                                       |                                                                            |
|---------------------------------------|----------------------------------------------------------------------------|
| LINFLEXD_UART_IP_STATUS_SUCCESS       | Generic operation success status.                                          |
| LINFLEXD_UART_IP_STATUS_ERROR         | Generic operation failure status.                                          |
| LINFLEXD_UART_IP_STATUS_BUSY          | Generic operation busy status.                                             |
| LINFLEXD_UART_IP_STATUS_TIMEOUT       | Generic operation timeout status.                                          |
| LINFLEXD_UART_IP_STATUS_TX_UNDERRUN   | TX underrun error.                                                         |
| LINFLEXD_UART_IP_STATUS_RX_OVERRUN    | RX overrun error.                                                          |
| LINFLEXD_UART_IP_STATUS_ABORTED       | <ul style="list-style-type: none"> <li>• A transfer was aborted</li> </ul> |
| LINFLEXD_UART_IP_STATUS_FRAMING_ERROR | Framing error.                                                             |
| LINFLEXD_UART_IP_STATUS_PARITY_ERROR  | Parity error.                                                              |
| LINFLEXD_UART_IP_STATUS_NOISE_ERROR   | Noise error.                                                               |
| LINFLEXD_UART_IP_STATUS_DMA_ERROR     | DMA error.                                                                 |

Definition at line 121 of file `Linflexd_Uart_Ip_Types.h`.

### 6.1.4.4 Linflexd\_Uart\_Ip\_EventType

enum `Linflexd_Uart_Ip_EventType`

Define the enum of the Events which can trigger UART callback.

This enum should include the Events for all platforms.

Enumerator

|                                     |                                    |
|-------------------------------------|------------------------------------|
| LINFLEXD_UART_IP_EVENT_RX_FULL      | Rx buffer is full.                 |
| LINFLEXD_UART_IP_EVENT_TX_EMPTY     | Tx buffer is empty.                |
| LINFLEXD_UART_IP_EVENT_END_TRANSFER | The current transfer is ending.    |
| LINFLEXD_UART_IP_EVENT_ERROR        | An error occurred during transfer. |

Definition at line 149 of file `Linflexd_Uart_Ip_Types.h`.

#### 6.1.4.5 Linflexd\_Uart\_Ip\_BaudrateType

enum `Linflexd_Uart_Ip_BaudrateType`

Baudrate values supported by the driver.

Definition at line 228 of file `Linflexd_Uart_Ip_Types.h`.

### 6.1.5 Function Reference

#### 6.1.5.1 Linflexd\_Uart\_Ip\_SetBaudrate()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_SetBaudrate (
    const uint8 Instance,
    const Linflexd_Uart_Ip_BaudrateType DesiredBaudRate,
    const uint32 ClockFrequency )
```

Sets the baud rate for UART communication.

This function computes the fractional and integer parts of the baud rate divisor to obtain the desired baud rate using the current protocol clock.

Parameters

|    |                        |                                                                              |
|----|------------------------|------------------------------------------------------------------------------|
| in | <i>Instance</i>        | - LINFLEXD instance number.                                                  |
| in | <i>DesiredBaudRate</i> | - Desired baud rate.                                                         |
| in | <i>ClockFrequency</i>  | - the current clock frequency used in the baud rate parameters calculations. |

Returns

`Linflexd_Uart_Ip_StatusType`

Return values

|                                        |                                                                               |
|----------------------------------------|-------------------------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>    | - A transfer is ongoing, therefore a baudrate value change can't be possible. |
| <i>LINFLEXD_UART_IP_STATUS_ERROR</i>   | - Error in changing the baudrate.                                             |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Baudrate value changed successfully.                                        |

#### 6.1.5.2 Linflexd\_Uart\_Ip\_GetBaudrate()

```
void Linflexd_Uart_Ip_GetBaudrate (
    const uint8 Instance,
    uint32 * ConfiguredBaudRate )
```

## Module Documentation

Gets the baud rate for UART communication.

This function returns the current UART baud rate, according to register values and the protocol clock frequency.

Parameters

|     |                           |                                                                                         |
|-----|---------------------------|-----------------------------------------------------------------------------------------|
| in  | <i>Instance</i>           | - LINFLEXD instance number.                                                             |
| out | <i>ConfiguredBaudRate</i> | - Pointer to a valid memory location where the current baudrate value will be provided. |

Returns

void

### 6.1.5.3 Linflexd\_Uart\_Ip\_Init()

```
void Linflexd_Uart_Ip_Init (  
    const uint8 Instance,  
    const Linflexd_Uart_Ip_UserConfigType * UserConfig )
```

Initializes a LINFLEXD instance for UART operations.

Parameters

|                       |                                 |
|-----------------------|---------------------------------|
| <i>Instance[in]</i>   | - LINFLEXD instance number.     |
| <i>UserConfig[in]</i> | - User configuration structure. |

Returns

void

### 6.1.5.4 Linflexd\_Uart\_Ip\_Deinit()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_Deinit (  
    const uint8 Instance )
```

Shuts down the UART functionality of the LINFLEXD module by disabling interrupts and transmitter/receiver.

Parameters

|                     |                             |
|---------------------|-----------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number. |
|---------------------|-----------------------------|

Return values

|                                        |                                                              |
|----------------------------------------|--------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_ERROR</i>   | - The current transfer processes are not finished completely |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation ended successfully.                              |

#### 6.1.5.5 Linflexd\_Uart\_Ip\_SetTxBuffer()

```
void Linflexd_Uart_Ip_SetTxBuffer (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize )
```

Sets the internal driver reference to the tx buffer.

This function can be called from the tx callback to provide the driver with a new buffer, for continuous transmission.

Parameters

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                          |
| <i>TxBuff[in]</i>   | - Source buffer containing 8-bit data chars to send. |
| <i>TxSize[in]</i>   | - The number of bytes to send.                       |

Returns

void

#### 6.1.5.6 Linflexd\_Uart\_Ip\_SetRxBuffer()

```
void Linflexd_Uart_Ip_SetRxBuffer (
    const uint8 Instance,
    uint8 * RxBuff,
    const uint32 RxSize )
```

Sets the internal driver reference to the rx buffer.

This function can be called from the rx callback to provide the driver with a new buffer, for continuous reception.

Parameters

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                             |
| <i>TxBuff[in]</i>   | - Source buffer containing 8-bit data chars to receive. |
| <i>TxSize[in]</i>   | - The number of bytes to receive.                       |



Returns

void

### 6.1.5.7 Linflexd\_Uart\_Ip\_AbortReceivingData()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_AbortReceivingData (  
    const uint8 Instance )
```

Terminates a non-blocking receive early.

Parameters

|                     |                            |
|---------------------|----------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number |
|---------------------|----------------------------|

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                                       |
|----------------------------------------|-----------------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation has been successfully ended or no operation was on-going. |
|----------------------------------------|-----------------------------------------------------------------------|

### 6.1.5.8 Linflexd\_Uart\_Ip\_AbortSendingData()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_AbortSendingData (  
    const uint8 Instance )
```

Terminates a non-blocking transfer(send) early.

Parameters

|                     |                            |
|---------------------|----------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number |
|---------------------|----------------------------|

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                                       |
|----------------------------------------|-----------------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation has been successfully ended or no operation was on-going. |
|----------------------------------------|-----------------------------------------------------------------------|

#### 6.1.5.9 Linflexd\_Uart\_Ip\_SyncSend()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_SyncSend (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize,
    const uint32 Timeout )
```

Sends data using LINFLEXD module in UART mode with polling method.

Blocking means that the function does not return until the transmission is complete.

Parameters

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                          |
| <i>TxBuff[in]</i>   | - Source buffer containing 8-bit data chars to send. |
| <i>TxSize[in]</i>   | - TxBuff size.                                       |
| <i>Timeout[in]</i>  | Timeout value in microseconds.                       |

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                               |
|----------------------------------------|---------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>    | - A trasfer is ongoing, therefore a new transfer can't begin. |
| <i>LINFLEXD_UART_IP_STATUS_ERROR</i>   | - Error in transmission.                                      |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation ended successfully.                               |
| <i>LINFLEXD_UART_IP_STATUS_TIMEOUT</i> | - Operation has timeout.                                      |

#### 6.1.5.10 Linflexd\_Uart\_Ip\_SyncReceive()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_SyncReceive (
    const uint8 Instance,
```

```
uint8 * RxBuff,
const uint32 RxSize,
const uint32 Timeout )
```

Sends data using LINFLEXD module in UART mode with polling method.

Blocking means that the function does not return until the transmission is complete.

Parameters

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                             |
| <i>RxBuff[in]</i>   | - Source buffer containing 8-bit data chars to receive. |
| <i>RxSize[in]</i>   | - RxBuff size.                                          |
| <i>Timeout[in]</i>  | Timeout value in microseconds.                          |

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                               |
|----------------------------------------|---------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>    | - A trasfer is ongoing, therefore a new transfer can't begin. |
| <i>LINFLEXD_UART_IP_STATUS_ERROR</i>   | - Error in reception.                                         |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation ended successfully.                               |
| <i>LINFLEXD_UART_IP_STATUS_TIMEOUT</i> | - Operation has timeout.                                      |

### 6.1.5.11 Linflexd\_Uart\_Ip\_AsyncReceive()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_AsyncReceive (
    const uint8 Instance,
    uint8 * RxBuff,
    const uint32 RxSize )
```

Starts data reception from the LINFLEXD module in UART mode with non-blocking method.

This enables an a-sync method for receiving data. When used with a non-blocking transmission, the UART driver can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the receive status to know when the receive is complete.

Parameters

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                               |
| <i>RxBuff[in]</i>   | - Buffer containing 8-bit read data chars to be received. |
| <i>RxSize[in]</i>   | - Size of RxBuff.                                         |

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                               |
|----------------------------------------|---------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>    | - A trasfer is ongoing, therefore a new transfer can't begin. |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation started successfully.                             |

#### 6.1.5.12 Linflexd\_Uart\_Ip\_AsyncSend()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_AsyncSend (
    const uint8 Instance,
    const uint8 * TxBuff,
    const uint32 TxSize )
```

Sends data using LINFLEXD module in UART mode with non-blocking method.

This enables an a-sync method for transmitting data. When used with a non-blocking receive, the UART driver can perform a full duplex operation. Non-blocking means that the function returns immediately. The application has to get the transmit status to know when the transmission is complete.

Parameters

|                     |                                                      |
|---------------------|------------------------------------------------------|
| <i>Instance[in]</i> | - LINFLEXD instance number.                          |
| <i>TxBuff[in]</i>   | - source buffer containing 8-bit data chars to send. |
| <i>TxSize[in]</i>   | - the number of bytes to send.                       |

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                        |                                                               |
|----------------------------------------|---------------------------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>    | - A trasfer is ongoing, therefore a new transfer can't begin. |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i> | - Operation started successfully.                             |

#### 6.1.5.13 Linflexd\_Uart\_Ip\_GetTransmitStatus()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_GetTransmitStatus (
    const uint8 Instance,
    uint32 * BytesRemaining )
```

Returns whether the status of the previous transmission.

Parameters

|                            |                                                                                                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------|
| <i>Instance[in]</i>        | - LINFLEXD instance number.                                                                               |
| <i>BytesRemaining[out]</i> | - Pointer to value that is populated with the number of bytes that have been sent in the active transfer. |

Note

In DMA mode, this parameter may not be accurate, in case the transfer completes right after calling this function; in this edge-case, the parameter will reflect the initial transfer size, due to automatic reloading of the major loop count in the DMA transfer descriptor.

Returns

Linflexd\_Uart\_Ip\_StatusType

Return values

|                                          |                                          |
|------------------------------------------|------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>      | - A transfer is ongoing.                 |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i>   | - Previous operation ended successfully. |
| <i>LINFLEXD_UART_IP_STATUS_ERROR</i>     | - Previous operation had a DMA error.    |
| <i>LINFLEXD_UART_IP_STATUS_ABORTED</i>   | - Previous transfer has been aborted     |
| <i>LINFLEXD_UART_IP_STATUS_DMA_ERROR</i> | - Previous transfer has DMA errors.      |

### 6.1.5.14 Linflexd\_Uart\_Ip\_GetReceiveStatus()

```
Linflexd_Uart_Ip_StatusType Linflexd_Uart_Ip_GetReceiveStatus (
    const uint8 Instance,
    uint32 * BytesRemaining )
```

Returns whether the status of the previous reception.

Parameters

|                            |                                                                                                                   |
|----------------------------|-------------------------------------------------------------------------------------------------------------------|
| <i>Instance[in]</i>        | - LINFLEXD instance number                                                                                        |
| <i>BytesRemaining[out]</i> | - Pointer to value that is filled with the number of bytes that still need to be received in the active transfer. |

## Note

The parameter BytesRemaining may not be accurate, in case the transfer completes right after calling this function; in this edge-case, this parameter will have the wrong value.

## Returns

Linflexd\_Uart\_Ip\_StatusType

## Return values

|                                              |                                            |
|----------------------------------------------|--------------------------------------------|
| <i>LINFLEXD_UART_IP_STATUS_BUSY</i>          | - A transfer is ongoing.                   |
| <i>LINFLEXD_UART_IP_STATUS_SUCCESS</i>       | - Previous operation ended successfully.   |
| <i>LINFLEXD_UART_IP_STATUS_RX_OVERRUN</i>    | - Previous operation had an overrun error. |
| <i>LINFLEXD_UART_IP_STATUS_FRAMING_ERROR</i> | - Previous operation had a framing error.  |
| <i>LINFLEXD_UART_IP_STATUS_DMA_ERROR</i>     | - Previous transfer has DMA errors.        |
| <i>LINFLEXD_UART_IP_STATUS_ABORTED</i>       | - Previous transfer has been aborted       |
| <i>LINFLEXD_UART_IP_STATUS_PARITY_ERROR</i>  | - A parity error occurred                  |
| <i>LINFLEXD_UART_IP_STATUS_NOISE_ERROR</i>   | - A noise error occurred                   |

## 6.2 UART Driver

### 6.2.1 Detailed Description

#### Data Structures

- struct [Uart\\_ChannelConfigType](#)  
*Uart channel configuration type structure. [More...](#)*
- struct [Uart\\_ConfigType](#)  
*Uart driver configuration type structure. [More...](#)*

#### Enum Reference

- enum [Uart\\_DrvStatusType](#)  
*Driver initialization status.*
- enum [Uart\\_DataDirectionType](#)  
*The type operation of an Uart channel.*
- enum [Uart\\_StatusType](#)  
*Uart operation status type.*

#### Function Reference

- void [Uart\\_Init](#) (const [Uart\\_ConfigType](#) \*Config)  
*Initializes the UART module.*
- void [Uart\\_Deinit](#) (void)  
*De-initializes the UART module.*
- Std\_ReturnType [Uart\\_SetBaudrate](#) (uint8 Channel, Uart\_BaudrateType Baudrate)  
*Configures the baud rate for the serial communication.*
- Std\_ReturnType [Uart\\_GetBaudrate](#) (uint8 Channel, uint32 \*Baudrate)  
*Retrieves the baud rate which is currently set for the serial communication.*
- void [Uart\\_SetBuffer](#) (uint8 Channel, uint8 \*Buffer, uint32 BufferSize, [Uart\\_DataDirectionType](#) Direction)  
*Configures a new buffer for continuous transfers.*
- Std\_ReturnType [Uart\\_SyncSend](#) (uint8 Channel, const uint8 \*Buffer, uint32 BufferSize, uint32 Timeout)  
*Starts a synchronous transfer of bytes.*
- Std\_ReturnType [Uart\\_SyncReceive](#) (uint8 Channel, uint8 \*Buffer, uint32 BufferSize, uint32 Timeout)  
*Starts a synchronous reception of bytes.*
- Std\_ReturnType [Uart\\_Abort](#) (uint8 Channel, [Uart\\_DataDirectionType](#) TransmissionType)  
*Aborts an on-going transfer.*
- Std\_ReturnType [Uart\\_AsyncSend](#) (uint8 Channel, const uint8 \*Buffer, uint32 BufferSize)  
*Starts an asynchronous transfer(send) of bytes.*
- Std\_ReturnType [Uart\\_AsyncReceive](#) (uint8 Channel, uint8 \*Buffer, uint32 BufferSize)  
*Starts an asynchronous transfer(receive) of bytes.*
- [Uart\\_StatusType](#) [Uart\\_GetStatus](#) (uint8 Channel, uint32 \*BytesTransferred, [Uart\\_DataDirectionType](#) TransferType)  
*Returns the status of the previous transfer.*

## 6.2.2 Data Structure Documentation

### 6.2.2.1 struct Uart\_ChannelConfigType

Uart channel configuration type structure.

This is the type of the external data structure containing the overall initialization data for one Uart Channel. A pointer to such a structure is provided to the Uart channel initialization routine for configuration of the Uart hardware channel.

Definition at line 317 of file Uart\_Types.h.

Data Fields

| Type                          | Name                  | Description                                         |
|-------------------------------|-----------------------|-----------------------------------------------------|
| uint8                         | UartChannelId         | Uart channel configured                             |
| uint32                        | ChannelClockFrequency | The clock frequency configured on the given channel |
| const Uart_Ipw_HwConfigType * | UartChannelConfig     | Pointer to a lower level channel configuration      |

### 6.2.2.2 struct Uart\_ConfigType

Uart driver configuration type structure.

This is the type of the pointer to the external data Uart Channels. A pointer of such a structure is provided to the Uart driver initialization routine for configuration of the Uart hardware channel.

Definition at line 339 of file Uart\_Types.h.

Data Fields

| Type                           | Name                        | Description                                                                                                                                                 |
|--------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| const Uart_ChannelConfigType * | Configs[UART_CH_MAX_CONFIG] | Hardware channel. Constant pointer of the constant external data structure containing the overall initialization data for all the configured Uart Channels. |

## 6.2.3 Enum Reference

### 6.2.3.1 Uart\_DrvStatusType

enum Uart\_DrvStatusType



Driver initialization status.

This enum contains the values for the driver initialization status.

Enumerator

|                 |                         |
|-----------------|-------------------------|
| UART_DRV_UNINIT | Driver not initialized. |
| UART_DRV_INIT   | Driver ready.           |

Definition at line 266 of file Uart\_Types.h.

### 6.2.3.2 Uart\_DataDirectionType

```
enum Uart_DataDirectionType
```

The type operation of an Uart channel.

Enumerator

|              |                          |
|--------------|--------------------------|
| UART_SEND    | The sending operation.   |
| UART_RECEIVE | The receiving operation. |

Definition at line 278 of file Uart\_Types.h.

### 6.2.3.3 Uart\_StatusType

```
enum Uart_StatusType
```

Uart operation status type.

Enumerator

|                               |                               |
|-------------------------------|-------------------------------|
| UART_STATUS_NO_ERROR          | Uart operation is successfull |
| UART_STATUS_OPERATION_ONGOING | Uart operation on going       |
| UART_STATUS_ABORTED           | Uart operation aborted        |
| UART_STATUS_FRAMING_ERROR     | Uart framing error            |
| UART_STATUS_RX_OVERRUN_ERROR  | Uart overrun error            |

Enumerator

|                          |                            |
|--------------------------|----------------------------|
| UART_STATUS_PARITY_ERROR | Uart parity error          |
| UART_STATUS_TIMEOUT      | Uart operation has timeout |
| UART_STATUS_NOISE_ERROR  | Uart noise error           |
| UART_STATUS_DMA_ERROR    | Uart Dma Error error       |

Definition at line 287 of file Uart\_Types.h.

## 6.2.4 Function Reference

### 6.2.4.1 Uart\_Init()

```
void Uart_Init (
    const Uart_ConfigType * Config )
```

Initializes the UART module.

This function performs software initialization of UART driver. It shall configure the Uart hardware peripheral for each channel.

Parameters

|    |               |                                             |
|----|---------------|---------------------------------------------|
| in | <i>Config</i> | - Pointer to UART driver configuration set. |
|----|---------------|---------------------------------------------|

Returns

void

Precondition

-

### 6.2.4.2 Uart\_Deinit()

```
void Uart_Deinit (
    void )
```

De-initializes the UART module.

This function performs software de-initialization of UART driver.

### Parameters

|   |  |
|---|--|
| - |  |
|---|--|

### Returns

void

### Precondition

-

#### 6.2.4.3 Uart\_SetBaudrate()

```
Std_ReturnType Uart_SetBaudrate (
    uint8 Channel,
    Uart_BaudrateType Baudrate )
```

Configures the baud rate for the serial communication.

This function performs the setting of the communication baudrate provided in the parameter.

### Parameters

|    |                 |                                 |
|----|-----------------|---------------------------------|
| in | <i>Channel</i>  | - Uart channel to be addressed. |
| in | <i>Baudrate</i> | - Baudrate value to be set.     |

### Returns

Std\_ReturnType.

### Return values

|                 |                                                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or a transfer is on-going or wrong core is addressed. |
| <i>E_OK</i>     | Successfull baudrate configuration.                                                                                      |

### Precondition

Uart\_Init function must be called before this API.

#### 6.2.4.4 Uart\_GetBaudrate()

```
Std_ReturnType Uart_GetBaudrate (
    uint8 Channel,
    uint32 * Baudrate )
```

Retrieves the baud rate which is currently set for the serial communication.

This function returns via the second parameter the current serial baudrate.

Parameters

|     |                 |                                                                      |
|-----|-----------------|----------------------------------------------------------------------|
| in  | <i>Channel</i>  | - Uart channel to be addressed.                                      |
| out | <i>Baudrate</i> | - Pointer to a memory location where the baudrate value is returned. |

Returns

Std\_ReturnType.

Return values

|                 |                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or a transfer is on-going or wrong core is addressed or a NULL_PTR pointer has been provided |
| <i>E_OK</i>     | Successfull baudrate retrieval.                                                                                                                                 |

Precondition

Uart\_Init function must be called before this API. Otherwise a random value can be returned.

#### 6.2.4.5 Uart\_SetBuffer()

```
void Uart_SetBuffer (
    uint8 Channel,
    uint8 * Buffer,
    uint32 BufferSize,
    Uart_DataDirectionType Direction )
```

Configures a new buffer for continuous transfers.

This function can be called inside a notification callback and offers the possibility to change the buffer in order to assure a continuous asynchronous transfer.

Parameters

|    |                |                                 |
|----|----------------|---------------------------------|
| in | <i>Channel</i> | - Uart channel to be addressed. |
|----|----------------|---------------------------------|

### Parameters

|    |                   |                                                                                                                                                                                                                                                                                                                                 |
|----|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in | <i>Buffer</i>     | - The new buffer provided.                                                                                                                                                                                                                                                                                                      |
| in | <i>BufferSize</i> | - The size of the new buffer.                                                                                                                                                                                                                                                                                                   |
| in | <i>Direction</i>  | - This parameter indicates for which type of transmission is the buffer set. Its values are UART_SEND for setting a buffer when the previous buffer is empty and there are more bytes to send and UART_RECEIVE to set a new buffer when the previous buffer is full with received buffer and there is more data to be received. |

### Returns

void.

### Precondition

Uart\_Init function must be called before this API.

#### 6.2.4.6 Uart\_SyncSend()

```
Std_ReturnType Uart_SyncSend (
    uint8 Channel,
    const uint8 * Buffer,
    uint32 BufferSize,
    uint32 Timeout )
```

Starts a synchronous transfer of bytes.

This function starts sending a number of bytes in a synchronous manner.

### Parameters

|    |                   |                                                   |
|----|-------------------|---------------------------------------------------|
| in | <i>Channel</i>    | - Uart channel to be addressed.                   |
| in | <i>Buffer</i>     | - The buffer which contains the bytes to be sent. |
| in | <i>BufferSize</i> | - The Buffer size.                                |
| in | <i>Timeout</i>    | - The timeout in us.                              |

### Returns

Std\_ReturnType.

Return values

|                 |                                                                                                                                                                                                                                                                                     |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong core has been accessed or a transfer is already on going on the requested channel or timeout occurred. |
| <i>E_OK</i>     | Successful transfer.                                                                                                                                                                                                                                                                |

Precondition

Uart\_Init function must be called before this API.

#### 6.2.4.7 Uart\_SyncReceive()

```
Std_ReturnType Uart_SyncReceive (
    uint8 Channel,
    uint8 * Buffer,
    uint32 BufferSize,
    uint32 Timeout )
```

Starts a synchronous reception of bytes.

This function starts receiving a number of bytes in a synchronous manner.

Parameters

|    |                   |                                               |
|----|-------------------|-----------------------------------------------|
| in | <i>Channel</i>    | - Uart channel to be addressed.               |
| in | <i>Buffer</i>     | - The buffer where the bytes will be located. |
| in | <i>BufferSize</i> | - The Buffer size.                            |
| in | <i>Timeout</i>    | - The timeout in us.                          |

Returns

Std\_ReturnType.

Return values

|                 |                                                                                                                                                                                                                                                                                      |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong core has been accessed or a reception is already on going on the requested channel or timeout occurred. |
| <i>E_OK</i>     | Successful reception.                                                                                                                                                                                                                                                                |

### Precondition

Uart\_Init function must be called before this API.

#### 6.2.4.8 Uart\_Abort()

```
Std_ReturnType Uart_Abort (
    uint8 Channel,
    Uart_DataDirectionType TransmissionType )
```

Aborts an on-going transfer.

This function aborts either a reception or a transmission depending on the last parameter.

### Parameters

|    |                         |                                                                                   |
|----|-------------------------|-----------------------------------------------------------------------------------|
| in | <i>Channel</i>          | - Uart channel to be addressed.                                                   |
| in | <i>TransmissionType</i> | - Type of the transfer to be aborted. It can be either UART_SEND or UART_RECEIVE. |

### Returns

Std\_ReturnType.

### Return values

|                 |                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or a wrong core has been accessed |
| <i>E_OK</i>     | Successful transfer aborted or in case no transfer was on going.                                     |

### Precondition

Uart\_Init function must be called before this API.

#### 6.2.4.9 Uart\_AsyncSend()

```
Std_ReturnType Uart_AsyncSend (
    uint8 Channel,
    const uint8 * Buffer,
    uint32 BufferSize )
```

Starts an asynchronous transfer(send) of bytes.

This function starts sending a number of bytes in an asynchronous manner. The transfer can be performed using either DMA or interrupts depending on the transfer type configured on the addressed channel.

## Parameters

|    |                   |                                                    |
|----|-------------------|----------------------------------------------------|
| in | <i>Channel</i>    | - Uart channel to be addressed.                    |
| in | <i>Buffer</i>     | - The buffer where the data to be sent is located. |
| in | <i>BufferSize</i> | - The Buffer size.                                 |

## Returns

Std\_ReturnType.

## Return values

|                 |                                                                                                                                                                                                                                                                       |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong core has been accessed or a transfer(send) is already on going on the requested channel. |
| <i>E_OK</i>     | The transfer(send) started successfully.                                                                                                                                                                                                                              |

## Precondition

Uart\_Init function must be called before this API.

**6.2.4.10 Uart\_AsyncReceive()**

```
Std_ReturnType Uart_AsyncReceive (
    uint8 Channel,
    uint8 * Buffer,
    uint32 BufferSize )
```

Starts an asynchronous transfer(receive) of bytes.

This function starts receiving a number of bytes in an asynchronous manner. The transfer can be performed using either DMA or interrupts depending on the transfer type configured on the addressed channel.

## Parameters

|    |                   |                                                          |
|----|-------------------|----------------------------------------------------------|
| in | <i>Channel</i>    | - Uart channel to be addressed.                          |
| in | <i>Buffer</i>     | - The buffer where the data to be received will located. |
| in | <i>BufferSize</i> | - The Buffer size.                                       |

## Returns

Std\_ReturnType.



Return values

|                 |                                                                                                                                                                                                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>E_NOT_OK</i> | If the Uart Channel is not valid or Uart driver is not initialized or Buffer is a NULL_PTR or BufferSize is 0, meaning no space has been allocated for the buffer or a wrong core has been accessed or a transfer(receive) is already on going on the requested channel. |
| <i>E_OK</i>     | The transfer(receive) started successfully.                                                                                                                                                                                                                              |

Precondition

Uart\_Init function must be called before this API.

### 6.2.4.11 Uart\_GetStatus()

```
Uart_StatusType Uart_GetStatus (
    uint8 Channel,
    uint32 * BytesTransferred,
    Uart_DataDirectionType TransferType )
```

Returns the status of the previous transfer.

This function returns the status of the previous transfer. If there is a transfer in progress, this function will also get the number of remaining bytes at the time the function was called.

Parameters

|     |                         |                                                                   |
|-----|-------------------------|-------------------------------------------------------------------|
| in  | <i>Channel</i>          | - Uart channel to be addressed.                                   |
| out | <i>BytesTransferred</i> | - A pointer where the number of remaining bytes will be written.  |
| in  | <i>TransferType</i>     | - The type of transfer in discussion (UART_SEND or UART_RECEIVE). |

Returns

Uart\_StatusType.

Return values

|                                      |                                                                                                              |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>UART_STATUS_NO_ERROR</i>          | - Operation has ended successfully.                                                                          |
| <i>UART_STATUS_FRAMING_ERROR</i>     | - Operation has had a framing error. This status is returned only if the TransferType parameter is RECEIVE.  |
| <i>UART_STATUS_RX_OVERRUN_ERROR</i>  | - Operation has had an overrun error. This status is returned only if the TransferType parameter is RECEIVE. |
| <i>UART_STATUS_PARITY_ERROR</i>      | - Operation has had a parity error. This status is returned only if the TransferType parameter is RECEIVE.   |
| <i>UART_STATUS_OPERATION_ONGOING</i> | - Operation has not finished at the moment of function call.                                                 |
| <i>UART_STATUS_ABORTED</i>           | - Operation has been aborted.                                                                                |

## Return values

|                            |                                                                                                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>UART_STATUS_TIMEOUT</i> | - Operation has had timeout in synchronous transfer functions with timeout value pass in Timeout parameter or functions that use the loop function whose execution time exceeds the timeout value configured through the UI |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## Precondition

Uart\_Init function must be called before this API.

**How to Reach Us:**

**Home Page:**

[nxp.com](http://nxp.com)

**Web Support:**

[nxp.com/support](http://nxp.com/support)

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: [nxp.com/SalesTermsandConditions](http://nxp.com/SalesTermsandConditions).

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

